

$$\frac{n-1}{2} (n-1)$$

Topic - 1

Searching (Imp For final)

Linear Search (Θ $O(n)$)

Given 'value' and array $a[]$, find index i such that $a[i] = \text{value}$, or report that no such index exists

* search space = place to search in. In this case, $a[]$ is search space

- * Best Case Condition: value in first index of search ($O(1)$) $O(\approx n)$
- * Average " : value in any index that's not in first or last ($\frac{n+1}{2}$) $O(n)$
- * Worst " : value in last index of search (Θ) $O(n)$

Time Complexity: Big $O(n)$
 " Θ (Θ)
 " $\Omega(\omega)$ } Asymptotic Notation
 ↓
 can be expressed in a number line in between 1 and n !

Random Search

Given . . . , randomly pick index i such that . . .

* Exploitation : reduction of search space

No replace

Best Case : 1

* Linear search is a variation of Random Search

Average " : $\approx n$

Worst " : n

Replace

Best case : 1

Average " : ∞

Worst " : ∞

Binary Search

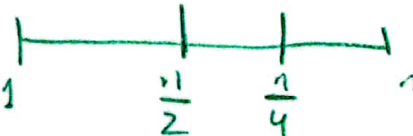
Given 'value' and sorted array . . .

Invariant : Maintains $a[\text{low}] \leq \text{value} \leq a[\text{high}]$

Best case : 1

Avg " :

Worst " : $\log_2(n)$

+  until you find it

Binary Search Algo

Bin_search (A, k) ^{Array} ^{value}

low = 0

high = size(A) - 1

while low ≤ high

mid = $\frac{low + high}{2}$

if A[mid] = k;

return found

else if A[mid] > k:

high = mid - 1

else

low = mid + 1

return not found

[replace {} with 1

* Algorithm is code without syntax

+ Binary search in an unsorted array is the same as replacing random array

+ For best case, worst case of B.S. we have

* use for any sorted thing

Pseudocode, Algorithm, Flowchart

* Pseudocode:

Artificial and informal language that helps to develop algorithms. Very similar to plain/everyday English
(Avoid syntaxes and mathematical operation symbols)

* Algorithm:

Series of instructions to accomplish a task; very close to programming language; more like steps involving notations; ordered sequence of steps

(Avoid syntaxes)

* Flowchart:

Graphical representation of the sequence of operations i.e. Algorithms

Example:

write an algorithm to determine a student's final grade and indicate whether the student is passing/failing. The final grade is calculated as the average of four marks.

Pseudocode:

1. input a set of 4 marks
2. calculate the average by summing and dividing by 4
3. If average is below 40, print fail
4. else print pass

Algorithm:

1. input $M1, M2, M3, M4$
2. $grade \leftarrow \frac{M1 + M2 + M3 + M4}{4}$
(used for assigning)
3. if $grade < 40$: (no then)
print fail
4. else
print pass

code:

1. $cin \gg M1 \gg M2 \gg M3 \gg M4;$
2. $double\ grade = (M1 + M2 + M3 + M4) / 4.0;$
3. $if (grade < 40.0) \{$
 $cout \ll "Fail" \ll endl;$
 $\}$
4. $else \{$
 $cout \ll "Pass" \ll endl;$
 $\}$

* 'class theke kisu ashbe na' ~ Sir "15 marks common from memory"

From an array find minimum value:

pseudocode:

1. take ~~min~~ as ~~first~~ first element of array
2. run a loop ~~of~~ for each element in array
3. if min is ^{greater} ~~less~~ than element, change ^{the value of} min to that element
4. else continue running the loop
5. print min

algorithm:

1. $\text{min} \leftarrow \text{arr}[0]$
2. ~~while~~ For $i = 0$ to size of arr :
 - if $\text{min} > \text{arr}[i]$:
 - $\text{min} \leftarrow \text{arr}[i]$
 - end if
 - ~~else~~
3. print min

code:

1. $\text{int min} = \text{arr}[0];$
2. for ($\text{int } i = 0; i < \text{sizeof(arr)}; i++$) {
 - if ($\text{min} > \text{arr}[i]$) {
 - $\text{min} = \text{arr}[i];$}
3. $\text{printf}(\text{"cout"} << \text{min} << \text{"end"});$

Selection Sort

- 1) Swap two numbers $\rightarrow$ Swap(A, B)
Built in C++ function
- 2) Find the minimum value from a vector of integers

Algorithm: 0. Find_Min(A):

1. $n = \text{size}(A)$ $\rightarrow$ find size of vector
 2. $\text{min} = A[0]$ $\rightarrow$ considering first term to be lowest
 3. $\text{pos} = 0$
 4. For $i = 1$ to $n-1$:
 5. if $A[i] < \text{min}$:
 6. $\text{min} = A[i]$
 7. $\text{pos} = i$
 8. end if
 9. end For
 10. print min
 11. print pos
- $\rightarrow$ compare and swap find min

③ Find index of min

W

starting index

20	29	72	98	13	87	66	52	51	36
0	13	72	98	29	87	66	52	51	36
assumption of next array									
1	13	29	98	72	87	66	52	51	36
2	13	29	36	72	87	66	52	51	98
3	13	29	36	51	87	66	52	72	98

4. 13 29 36 51 52 66 87 72 98,

5. 13 29 36 51 52 66 87 72 98,

6. 13 29 36 51 52 66 72 87 98,

7. 13 29 36 51 52 66 72 87 98

Algorithm Pasing

* For n terms, sort has to be done $n-1$ times

Pasing $\rightarrow$ ~~Iteration~~ Iteration / Tracing of

Selection Sort Algorithm:

0. Selection - Sort(A):

1. $n = \text{size}(A)$ $\rightarrow$ $\leftarrow$

2. For $j = 0$ to $n-2$:

$\text{min} = A[j]$

3. $\text{pos} = j$

4. For $i = j+1$ to $n-1$:

5. if $A[i] < \text{min}$:

6. $\text{min} = A[i]$

7. $\text{pos} = i$

8. end if

9. end For

10. swap $A[\text{pos}], A[j]$ $\rightarrow$ do NOT use min

11. end For

12. end For

Complex. (min)

Best : n

Avg : n

Worst : n

Complexity of ^{Selection} Algo Sorting:

Best : $n(n-2) = n^2 - 2n$

Avg : $n(n-2) = "$

Worst : $n(n-2) = "$

C-3(w-2)

16/04/24

Insertion Sort

* Insert an element into the sorted array

$A = [10 \mid 15 \mid 20 \mid 25 \mid 30]$

Say, we want to insert 18,

Two different ways to achieve this $\longrightarrow$

- (1) ~~the~~ Search, then shift $\longrightarrow$ Complexity (n) : numbers of search + no. shift
- (2) search + Shift

* Shift from last element

the Search ~~and~~ then shift:

$A = [10 \mid 15 \mid 18 \mid 20 \mid 25 \mid 30]$

Search and Shift:

use $j--$ to navigate

if insert $\geq$ index, $A[\text{index}]$

$A[\text{index}] =$

$A[\text{index} + 1] = A[\text{index}]$ insert

else

$A[\text{index} + 1] = A[\text{index}]$

Algorithm (Insert): → insert element

0. Insert(A , key):

1. $n = \text{size}(A)$

2. $j = n - 1$

3. while $j \geq 0$ and $A[j] > \text{key}$

4. $A[j+1] = A[j]$

5. $j = j - 1$

6. end while

7. $A[j+1] = \text{key}$

Insertion Sort:

0.

No.	1	2	3	4	5	6
	5	2	4	6	1	3

End(n)

0	-1	2	3	4	5
---	----	---	---	---	---

0.

2	5	4	6	1	3
---	---	---	---	---	---

1.

2	4	5	6	1	3
---	---	---	---	---	---

2.

2	4	5	6	1	3
---	---	---	---	---	---

3.

1	2	4	5	6	3
---	---	---	---	---	---

4.

1	2	3	4	5	6
---	---	---	---	---	---

$n - 1$

Algorithm (Insertion Sort):

0. Insertion_Sort(A):

```
1.   n = size(A)
2.   For i = 1 to n-1: // starts from 1 NOT 0
3.       key = A[i] // consider the i-th term to be
                     inserted
4.       j = i-1 // to work on the prev indexes
5.       while j ≥ 0 and A[j] > key:
6.           A[j+1] = A[j] // shift
7.           j = j-1 // for the loop
8.       end while
9.       A[j+1] = key // if the location is found;
                     [the element is
                     greater than the index]
10.  end For
```

Complexity:

Best : n

Avg : n^2

Worst : n^2

* Binary search IS an improvement
↳ Do at Home (for Cut)

→ $n \log_2 n$

C-4(w-3)

22/04/24

Recursion:

① Definition of Recursion = see the definition of Recursion

② $n = 5$; 5 ^{1 2 3 4 5} 4 3 2 1 ②

for (int i = 0; i $\leq$ 5; i++) $\rightarrow$ iterative solution

{
 cout << i << " "; // do something

}

void

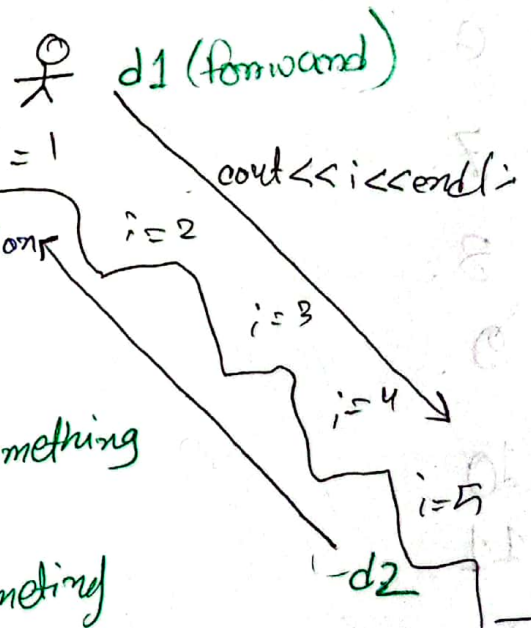
③ FOR (int i, int n) $\rightarrow$ recursive solutions

{

if (i <= n)

{ $\rightarrow$ d1 cout << i << " "; // do something
 FOR (i+1, n);

} $\rightarrow$ d2 cout << i << " "; // do something
}



* Useful for [dynamic programming] $\rightarrow$ saves every state of the program
 $\hookrightarrow$ params become "states"

int main()

{

FOR (1, 5);

}

Recursion Steps:

(2/2)

1. call FOR_1 with $i = 1$
2. call FOR_2 with $i = 2$
3. call FOR_3 with $i = 3$
4. call FOR_4 with $i = 4$
5. call FOR_5 with $i = 5$
6. call FOR_6 with $i = 6$
7. break ; no more calls
8. return to FOR_5
 print 5
9. return to FOR_4
10. return to FOR_4
11. print 4
12. return to FOR_3
13. print 3
14. return to FOR_2
15. print 2
16. return to FOR_1
17. print 1
18. return to main

* Reversing an array :

```
void FOR (int i, int n) // (int i, n)
{
```

```
    if (i >= 0) // (i < n)
    {
```

```
        cout << i << " "; // nothing
```

```
        FOR(i-1, i) // (i+1, n)
```

```
    }
    // cout << i << " ";
```

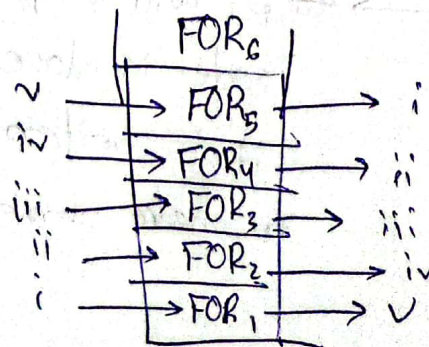
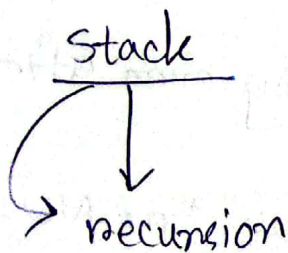
```
}
```

```
int main()
{
```

```
    FOR(n-1, 0) // FOR(0, n-1)
```

```
}
```

* Memory where recursion is kept
is called STACK MEMORY



L-3(w-3)

Pointers

23/04/24

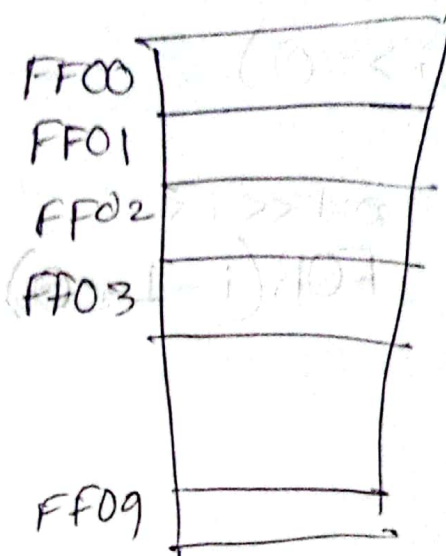
→ important
for linked list.

int x = 5

variable : x

value : 5

Address : FF00



int x = 5

int *p } int *p
p = &x }

*p = 25

≡ &x = 25

≡ x = 25

*After declaring p = &x

+ int *p = NULL } → p will keep pointing even after
p = &x code is closed
∴ it is better to use *p = NULL
to make sure address is not assigned

* $\text{int } *p = \text{arr}; \equiv \text{int } *p = \&\text{arr}[0]$

$\text{arr} = \{ 10, 12, 13, 14, 15 \}$

$\text{arr}[0] \longrightarrow 10$

$\&\text{arr}[0] \longrightarrow \text{address of } 10$

$*(\&\text{arr}[0]) \longrightarrow 10$

$*(&\text{arr}[0]) + 1 \longrightarrow 11 \quad // \quad 10 + 1$

$*(&\text{arr}[0] + 1) \longrightarrow 12 \quad // \quad \text{addr. of } 10 + 1$

$\hookrightarrow$

* Good practice to free the pointers

* Cpp allows method/function overloading, overriding etc

* Cpp is OOP ($\therefore$ better than java)

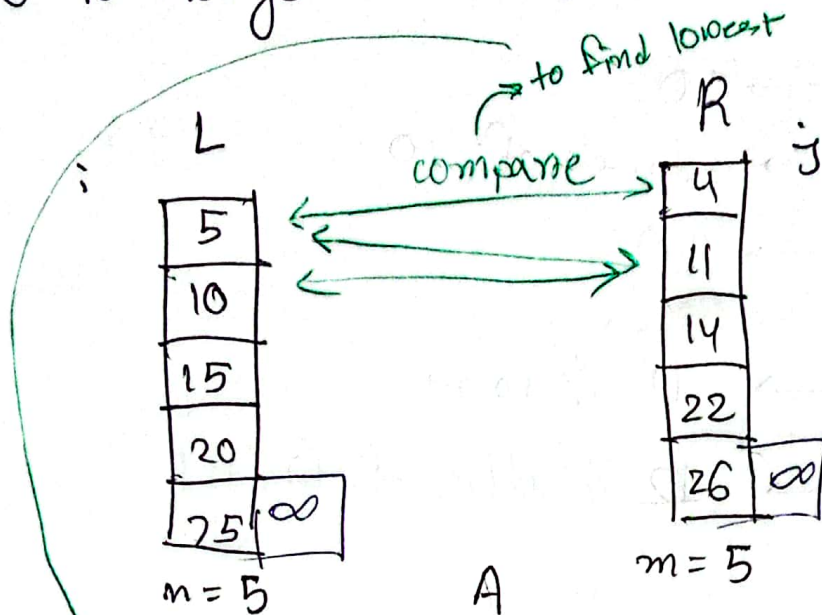
C-5(w-3)

Merge Sort: Merge

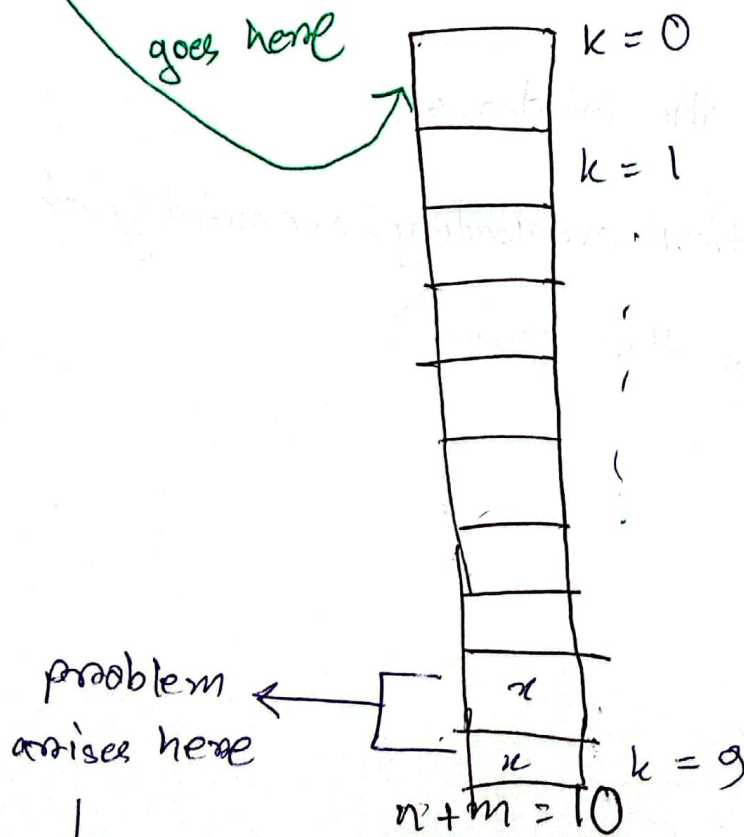
23/04/24

→ Conquer

* How to merge two sorted lists into one sorted list



Comparing:
 $L[i]$ $R[j]$



→ same value
will ~~happ~~ be placed
here twice

solⁿ → add index
+ put infinity then

#Algorithm for merging:

0. MERGE(L, R)

1. $m = \text{size}(L)$

2. $n = \text{size}(R)$

3. ~~$i = j = 0$~~ $L[m] = \infty$ // } ~~get~~ adding infinity at the end

4. $R[n] = \infty$ //

5. $i = j = 0$

6. For $k = 0$ to $(m+n) - 1$:

7. if $L[i] \leq R[j]$:

8. $A[k] = L[i]$

9. $i = i + 1$

10. ~~end if~~

11. ~~else if~~ ~~else~~

12. $A[k] = R[j]$

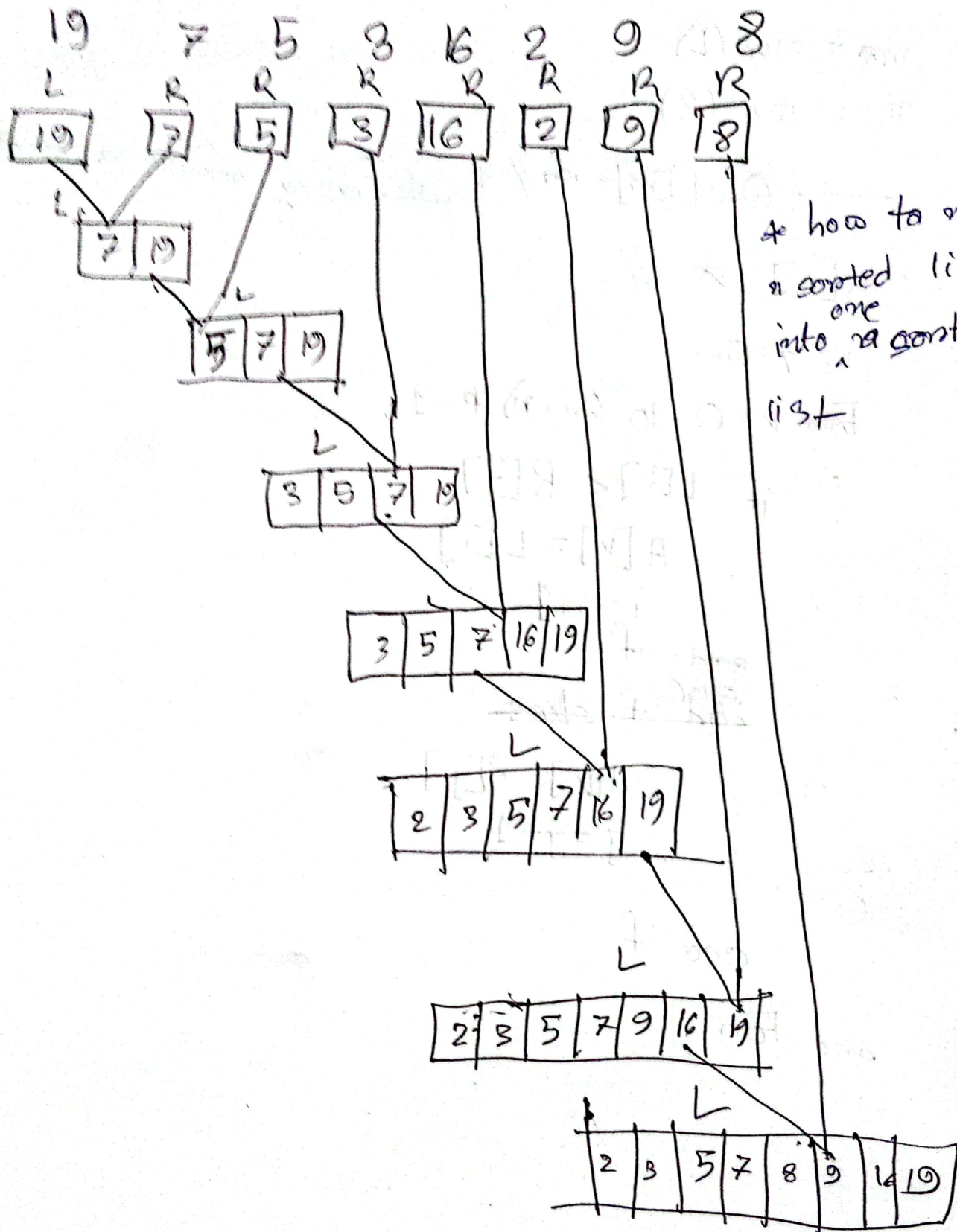
13. $j = j + 1$

14. ~~end if~~

15. end if

16. end For

* MERGE SORT



* how to merge
n sorted lists
into ^{one} sorted
list

*Algorithm (one ascending, one descending)

$L \rightarrow$ ascending / $R \rightarrow$ descending

MERGE(L, R)

$m = \text{size}(L)$

$n = \text{size}(R)$

$L[m] = \infty$

// $R[n] = -\infty \rightarrow$ do not use

$i = 0$

$j = n - 1$

For $k = 0$ to $(m+n)-1$:

if $L[i] \leq R[j]$:

$A[k] = L[i]$

$i = i + 1$

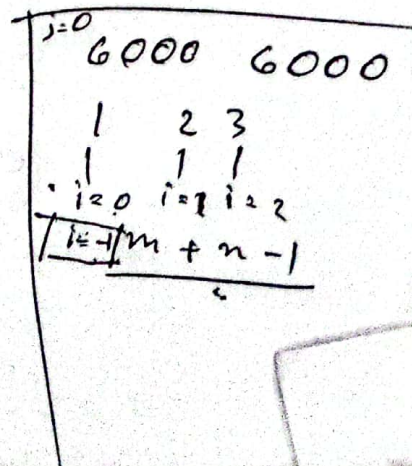
else:

$A[k] = R[j]$

$j = j - 1$

end if

end For



414:

0. MERGE (L, R):

1. $m = \text{size}(L)$

2. $n = \text{size}(R)$

$k = 0$

3. $i = 0$

$j = n - 1$

while $i \leq m$ and $j \geq 0$:

if $L[i] \leq R[j]$:

$A[k] = L[i]$

$i = i + 1$

$k = k + 1$

else:

$A[k] = R[j]$

$j = j - 1$

$k = k + 1$

end if

end while

while $i < m$:

$A[k] = L[i]$

$k = k + 1$

$i = i + 1$

end while

while $j \geq 0$:

$A[k] = R[j]$

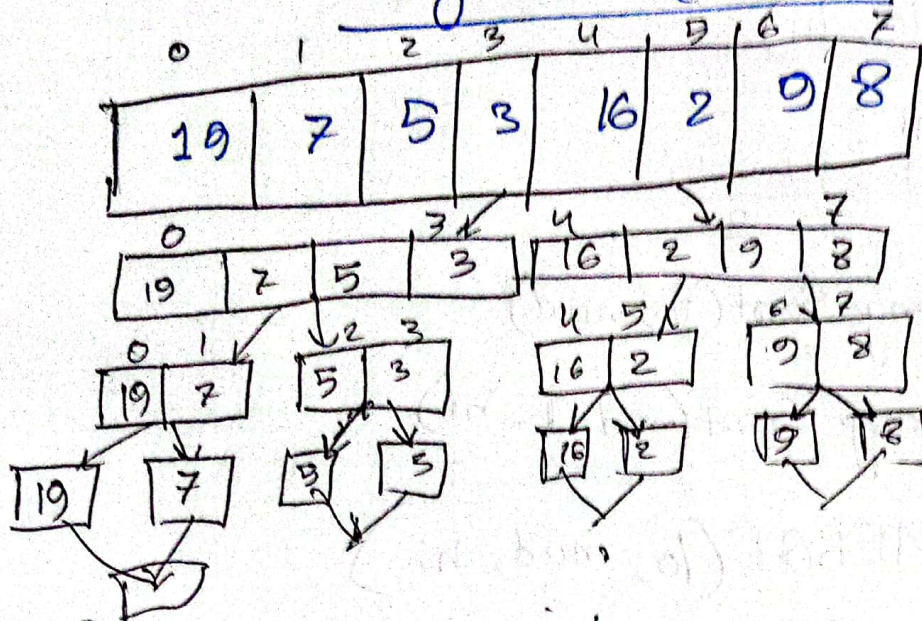
$k = k + 1$

$j = j - 1$

C-6(w-4)

29/04/24

Merge Sort (Divide)



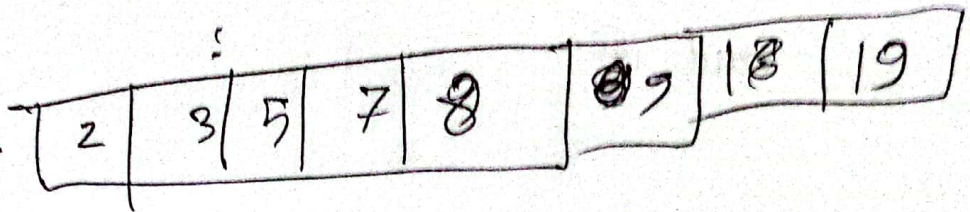
$$low = 0$$

$$hi = 7$$

$$mid = \frac{low + hi}{2}$$

$$= \frac{0 + 7}{2}$$

$$= 3$$



* Algorithm:

Complex: $n \log_2 n$

0. ~~ME~~ MergeSort(lo, hi)

1. if $lo < hi$

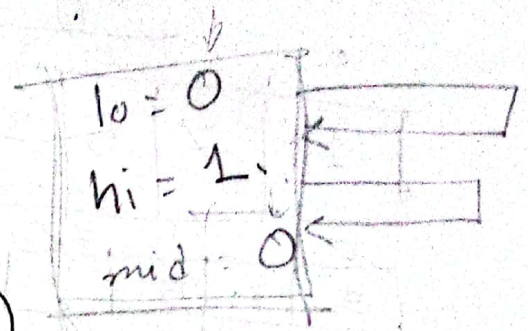
2. $mid = \frac{lo + hi}{2}$

3. MergeSort(lo, mid)

4. MergeSort($mid+1, hi$)

5. ~~end if~~ MERGE(lo, mid, hi)

6. end if



0. MERGE(lo, mid, hi):

1. $m = mid - lo + 1$

2. $n = hi - mid$

3. Let, $L[0 \dots m-1]$ and $R[0 \dots n-1]$ be new arrays

4. For $i = 0$ to $m-1$:

5. $L[i] = A[lo + i]$

6. end For

7. For $j = 0$ to $n-1$:

8. $R[j] = A[mid + 1 + j]$

9. end For

10. $L[m] = \infty, R[n] = \infty$

11. $i = j = 0$

12. For $k = 0$ to $(~~hi~~ ^{hi}) - 1$:

13. if $L[i] \leq R[j]$:

14. $A[k] = L[i]$

15. $i = i + 1$

16. ~~end if~~ else:

17. $A[k] = R[j]$

18. $j = j + 1$

19. end if

20. end For

$0 \rightarrow m + n - 1$
 $lo \rightarrow hi$

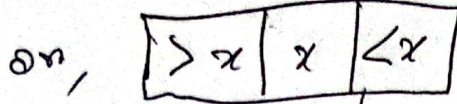
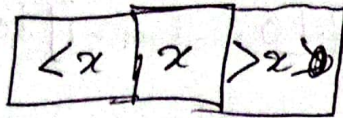
C++/L-4

Quick Sort

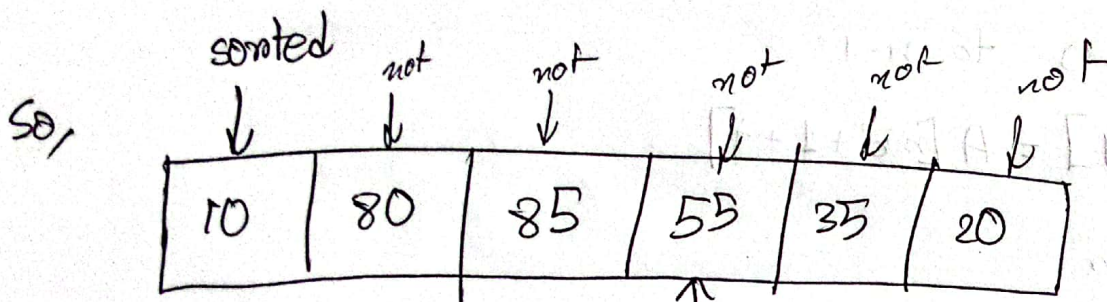
30/4/24

[Online]

A number is sorted when, —



or, $\rightarrow$ all elements in $> x$ and $< x$ being sorted by themselves is not a requirement

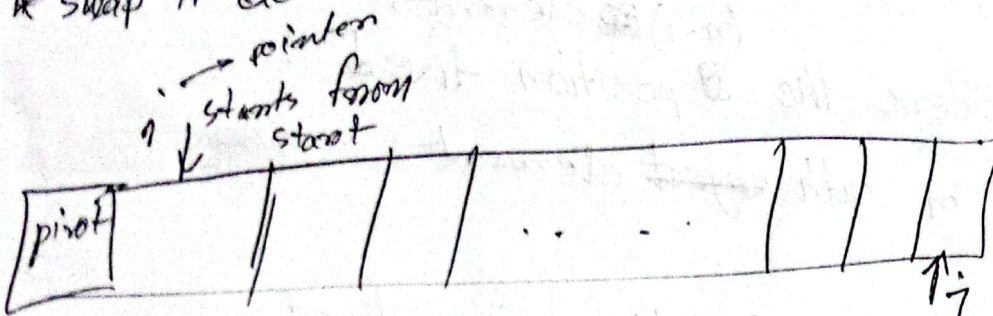


$\uparrow$
might be in its position but its not correct according to the definition

* To find the sorted position

Pivot $\rightarrow$ element in its sorted position
 $\rightarrow$ can be
 * Swap if element is not in position

$\rightarrow$ start
 $\rightarrow$ end
 $\rightarrow$ mid
 $\rightarrow$ random



$\uparrow$ j starts from end
 $\hookrightarrow$ pointer

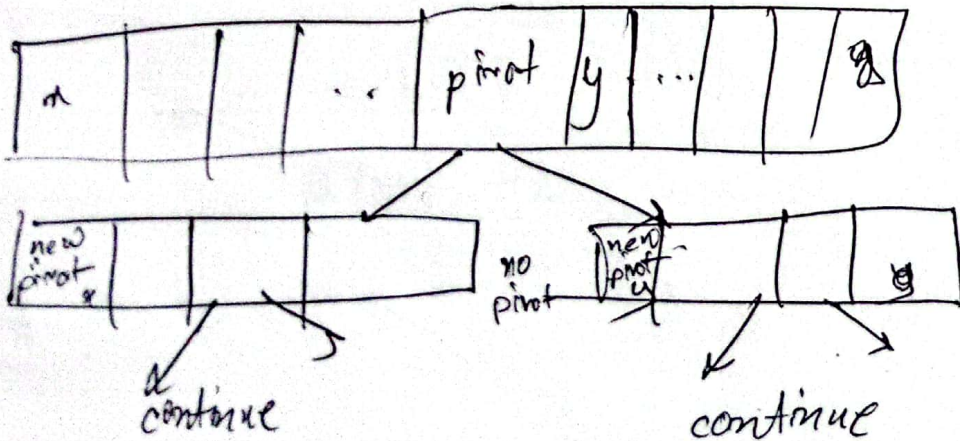
$i++$ $\rightarrow$ happens when $\leq \text{pivot}$
 $\rightarrow$ until $\geq \text{pivot}$

$j--$ $\rightarrow$ happens until $\leq \text{pivot}$
 $\rightarrow$ until $\geq \text{pivot}$

$\text{swap}(i, j) \rightarrow$ swapping between small and big

when $j < i$,

$\text{swap}(\text{pivot}, j) \rightarrow$ partitioning
 $\hookrightarrow$ swapping between ~~be~~ pivot and j



* Diff betⁿ merge and quick

↓ ↓
1. based on index 1. based on pivot

Selection Sort → Considers the $(n-1)$ elements
Picks the i th smallest element

Quick Sort → Considers the i th position fixed
Picks the i th element

* Algorithm

```
0. Partition(lo, hi):  
1.   pivot = A(lo)  
2.   i = lo // starts from lowest index  
3.   j = hi // " " " highest index  
4.   while i < j: // will run until they don't cross  
5.       wh while wh A[i] ≤ pivot:  
6.           i = i + 1 // to find big number  
7.       end while  
8.       while A[j] ≥ pivot:  
9.           j = j j - 1 // to find small number  
10.      end while  
11.      if i < j:  
12.          swap A[i], A[j]  
13.      end if  
14.  end while  
15.  swap A[lo], A[j]  
16.  return j
```

0. QuickSort(lo, hi):

if $lo < hi$:

1. $p = \text{Partition}(lo, hi)$

2. QuickSort($lo, p-1$) // left side

3. QuickSort($p+1, hi$) // right side

4.

Complexity:

Partition: $O(n)$

Recursive Breaking Down: $O(\log_2 n)$

$\therefore \text{QuickSort} = \cancel{n \log_2 n}$ Best: $n \log_2 n$

Avg: $n \log_2 n$

Worst: n^2

☐ Linked list can be defined as a collection of data items that can be stored in scattered memory locations. Here, data items must be linked with each others. Data items are known as ~~node~~ nodes. the

* A linear linked list (singly) is a list where there is only one way link between nodes.

* A doubly linked list is a list where there are links in both directions between nodes.

* A circular linked list is a list where the last node has a pointer which points to the first node

☐ Each type of linked list requires an external pointer to point the first node of the list (root)



insert

├ first
├ last
└ anywhere

delete

├ first
├ last
└ anywhere

traverse

├ print
├ search
└ others,

create

├ list/structure
├ declaration
└ pointer declaration

```
#include
using
```

```
struct node
{
```

```
    int data;
```

```
    node *address;
```

```
};
```

```
node *root = NULL;
```

```
int main()
```

```
{
    node *A, *B, *C;
```

A = new node();

B = new node();

C = new node();

A->data = 10; A->data = 10;

B->data = 20; B->data = 20;

C->data = 30; C->data = 30;

A->address = B

B->address = C

C->address = NULL

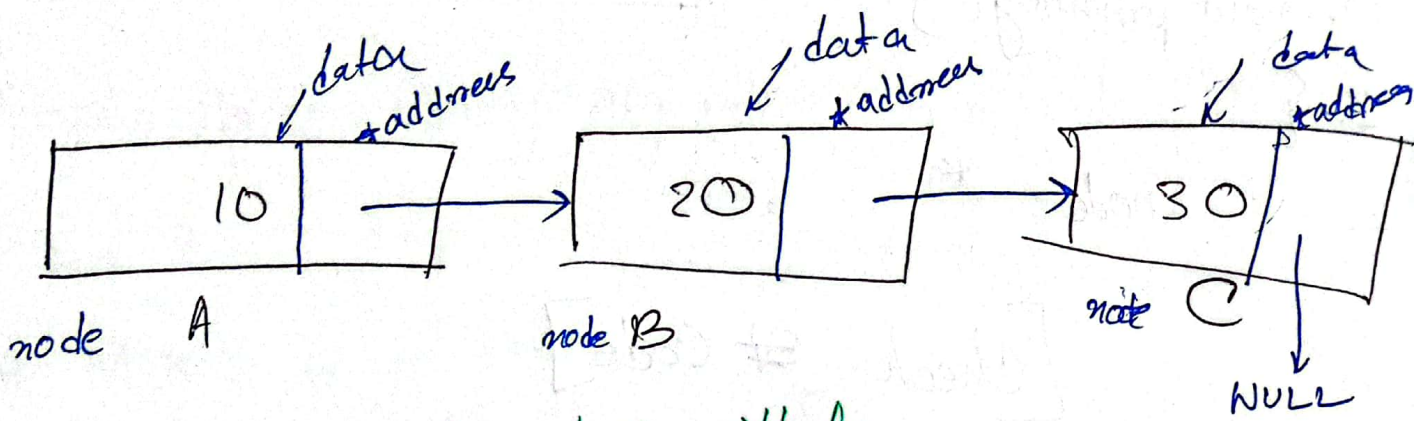
cout << A->data << " " << B->data << " " << C->data;

root = A;

cout << A->data << " " << A->address->data << " " << A->address->address->data;

<< A->address->address->address->data;

↳ shanu, error



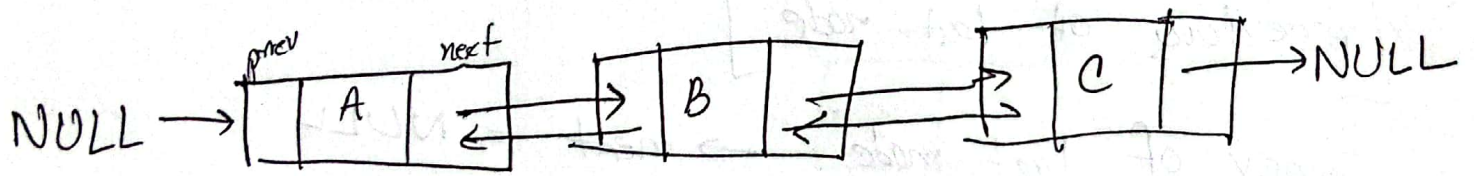
* We want to call B and C with A

* ~~Not poss!~~ Does not have same sequential addresses

* A will have address of B

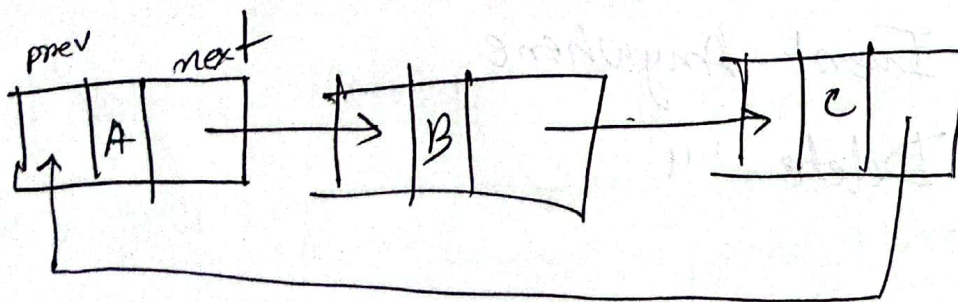
* data type will be struct for struct pointers

* Cpp uses $\rightarrow$ for pointer struct.



Doubly

Doubly



0. void printing()

1. {

2. node *

[Check ~~st~~ Code]

* Always assume list is ready

* If inserted at first, the list is ~~also~~ automatically reversed.

void delete last ()

{

[previous of last node]

prev of last node → next = NULL

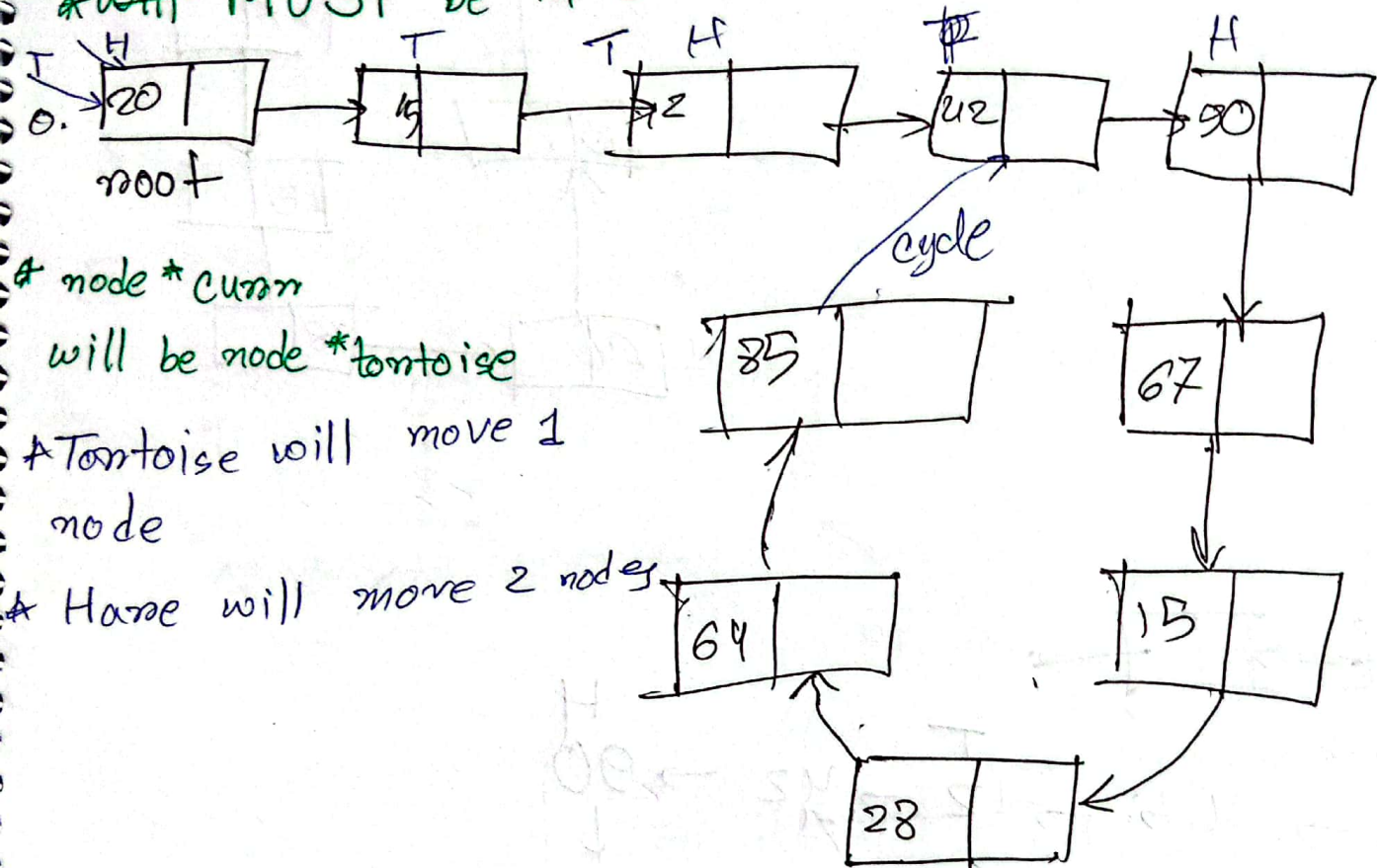
}

* Assignment: Insert Anywhere

Delete " "

Floyd's Cycle Finding Algorithm ('Hare and Tortoise Algorithm')

~~will~~ MUST be in exam



* node * curr

will be node * tortoise

* Tortoise will move 1 node

* Hare will move 2 nodes

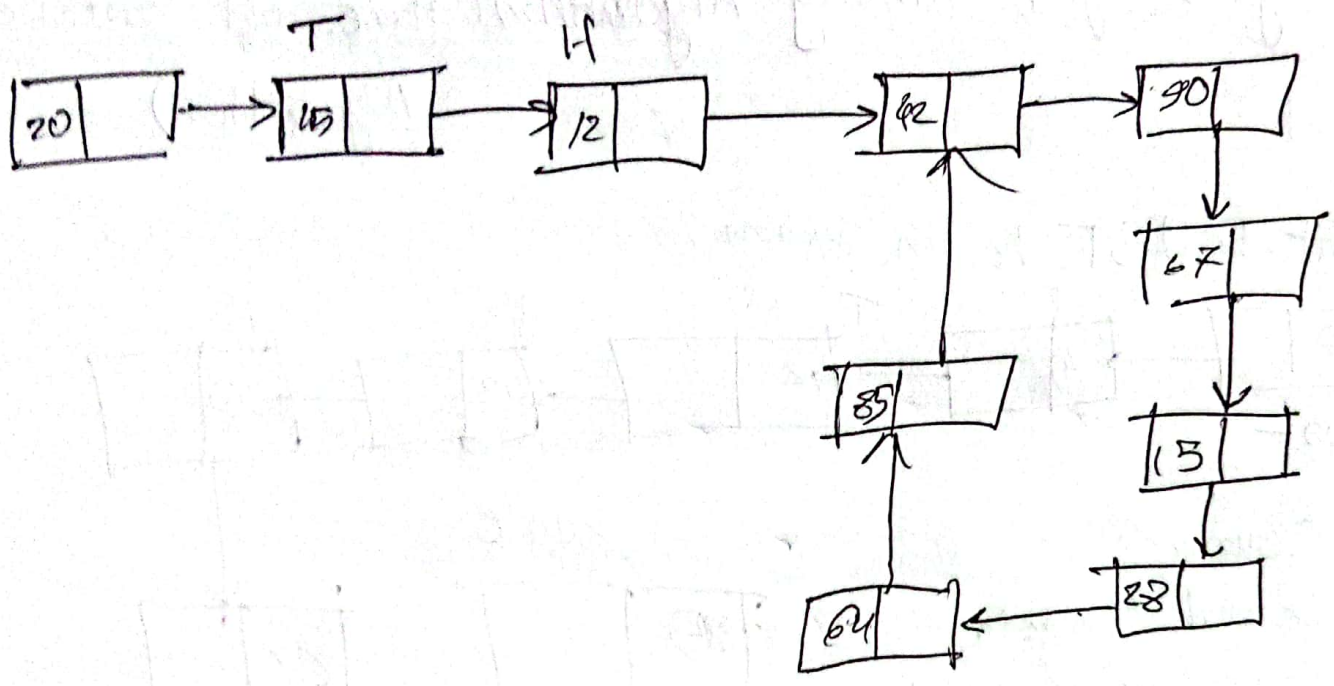
* Imp for job interview

* If starting node and ending nodes are distinct then it is ~~a~~ will not be cyclic

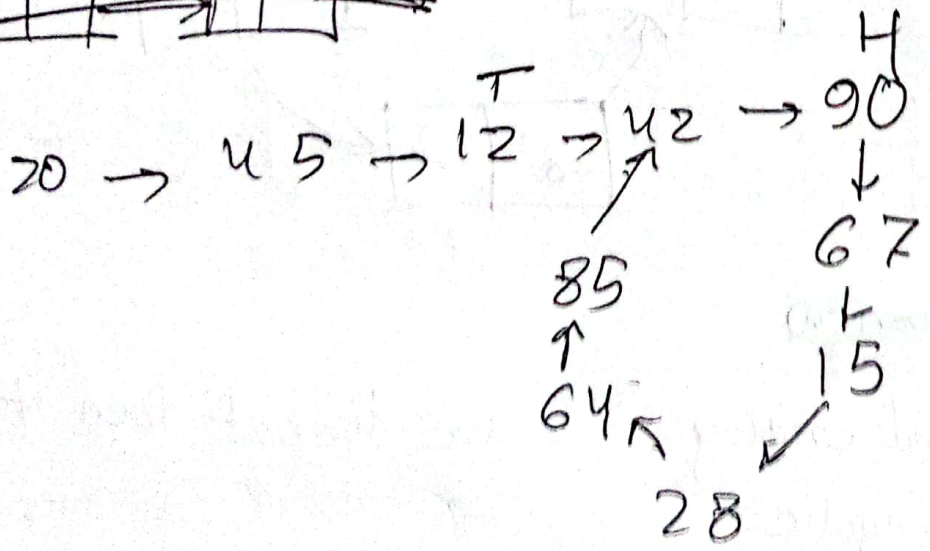
* To check, you can
{

 → the Map nodes
 → Compare address

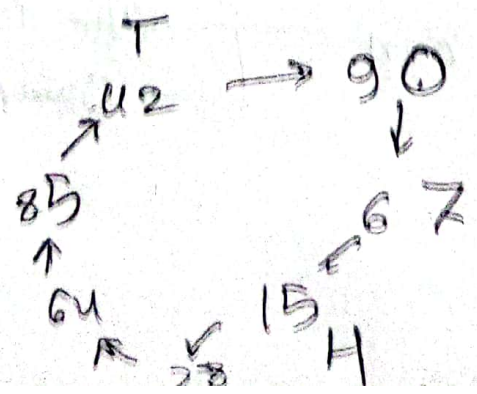
1.

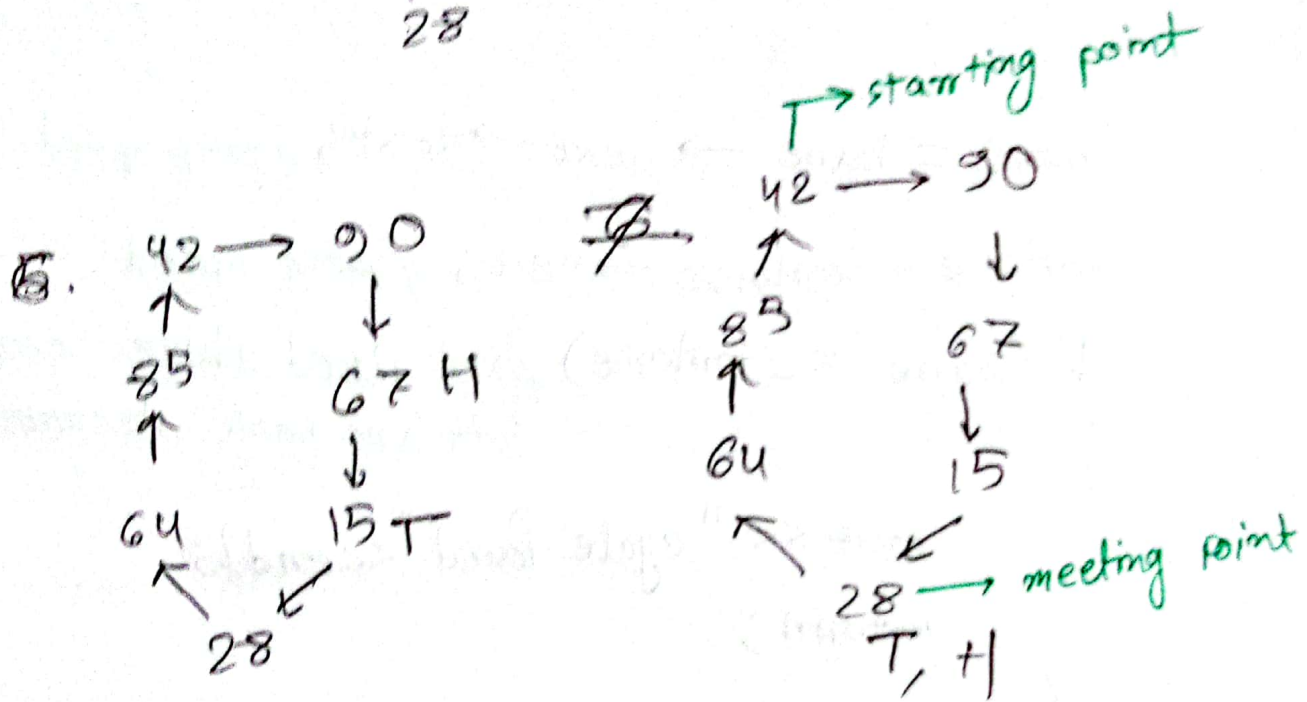
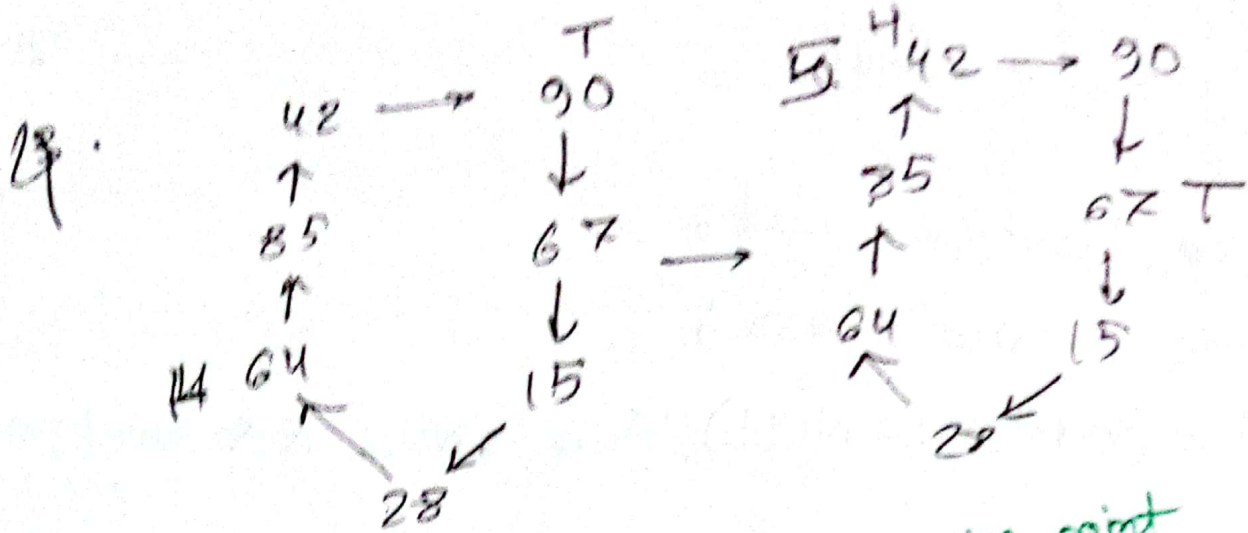


2.



20 3





T and H will meet if and only if one runs at the other's multiple speed

```
void cycle_ void find()
```

```
{
```

```
node *tortoise = root;
```

```
node *hare = root;
```

```
while (tortoise != NULL) // (hare != NULL && hare->next != NULL)
```

```
{
```

```
    hare = hare->next->next; // jump speed
```

```
    tortoise = tortoise->next; // move speed
```

```
    if (hare == tortoise) // Not placed above because  
                           both are roots at start
```

```
{
```

```
    cout << "cycle found" << endl;
```

```
    return;
```

```
}
```

```
}
```

```
    cout << "cycle not found" << endl;
```

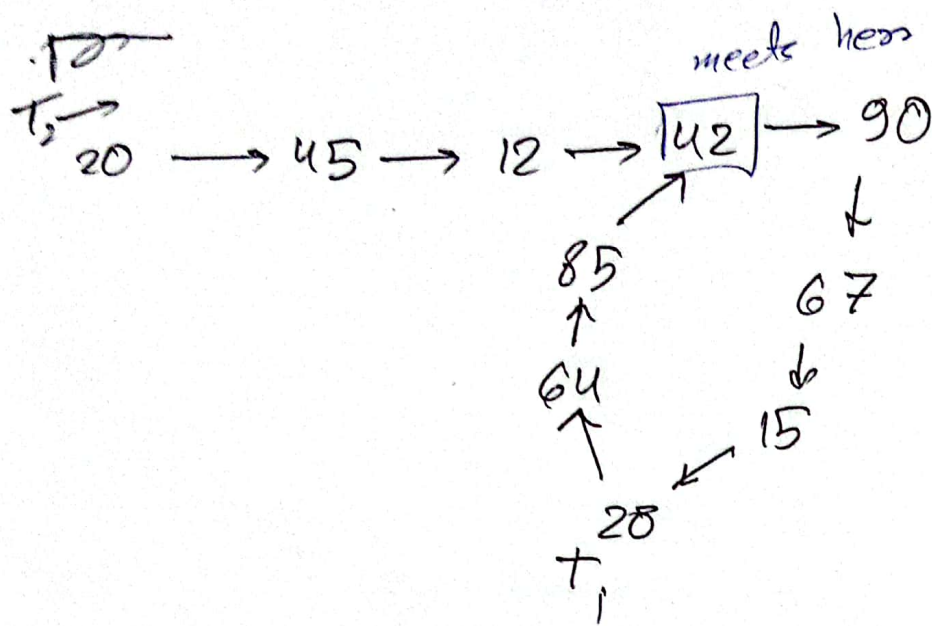
```
}
```

* Complexity : $O(m * n)$

+ snafayet's planet

* Multiple of each other

* To find meeting point



void starting - node()

{

node *tortoise = root;

while(

if (have == tortoise)

{

cout << "cycle found" << endl;

}

}

node *tortoise2 = root;

while(tortoise != tortoise2)

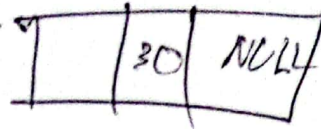
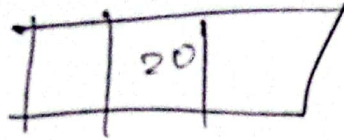
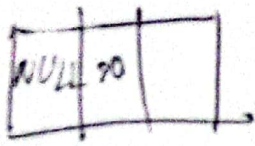
{

tortoise = tortoise -> next

tortoise2 = tortoise2 -> next

cout << "cycle found" << endl;

Doubly Linked List



Insert anywhere:

$\text{curr_node} \rightarrow \text{next} = \text{temp}$

$\text{temp} \rightarrow \text{next} = \text{posn_node}$

+ do prev as well

Delete anywhere:

$\text{agen_node} \rightarrow \text{next} = \text{posn_node}$

$\text{posn_node} \rightarrow \text{prev} = \text{agen_node}$

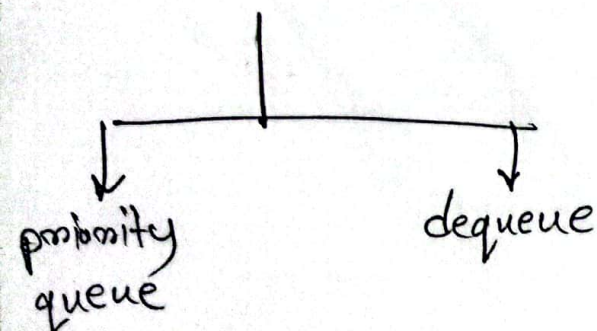
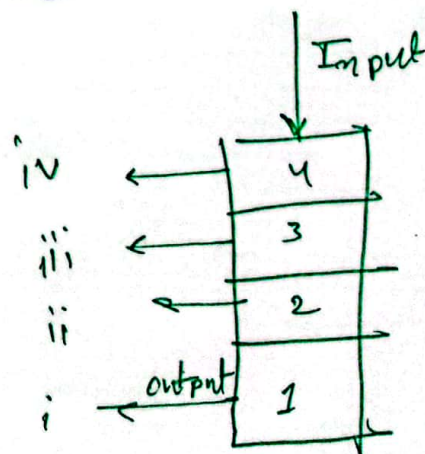
Stack and Queue

— Stack is a linear list where any element is added at the top of the list and any element is deleted/accessed from the top of the list. Add operation for a stack is called 'push' operation and deletion operation is called 'pop' operation. Stack is LIFO (Last In First Out) data structure. That means the element which was added will be deleted/accessed first.



Example: Scrolling/Browsing/Undo+Redo

— Queue is a linear list where all additions are made at one end, called 'rear' and all ~~data~~ deletions/accesses are made from another end called 'front' of the list. Que is a FIFO (First In First Out) data structure. That means the element that is added first will be deleted/accessed first.



57. Standard template library

#include _____
using namespace _____



int main()

{

stack<int> s; // container

s.push(10);

s.push(20);

s.push(30);

s.push(40);

cout << s.size() << endl;

while (true) // while (!s.empty())

{

if (s.size() > 0)

{

int val = s.top();

cout << val << endl;

s.pop();

}

}

}

→ 40
30
20
10

0/0/0/0/0

Stack using Linked List.

push() $\longrightarrow$ insert First()

top() $\longrightarrow$ root $\longrightarrow$ data

pop() $\longrightarrow$ delete First()

size() $\longrightarrow$ size() // count ++

empty() $\longrightarrow$ size() // count == 0

* if deleted inserted last, deleted last

Queue.

push() $\longrightarrow$ insert Last()

front() $\longrightarrow$ root $\longrightarrow$ data

pop() $\longrightarrow$ delete First()

size() $\longrightarrow$ size() // count ++

empty $\longrightarrow$ size() // count == 0

STL

Queue

#include

using namespace

int main()

{

queue<int> s;

s.push(10);

s.push(20);

while(!s.empty())

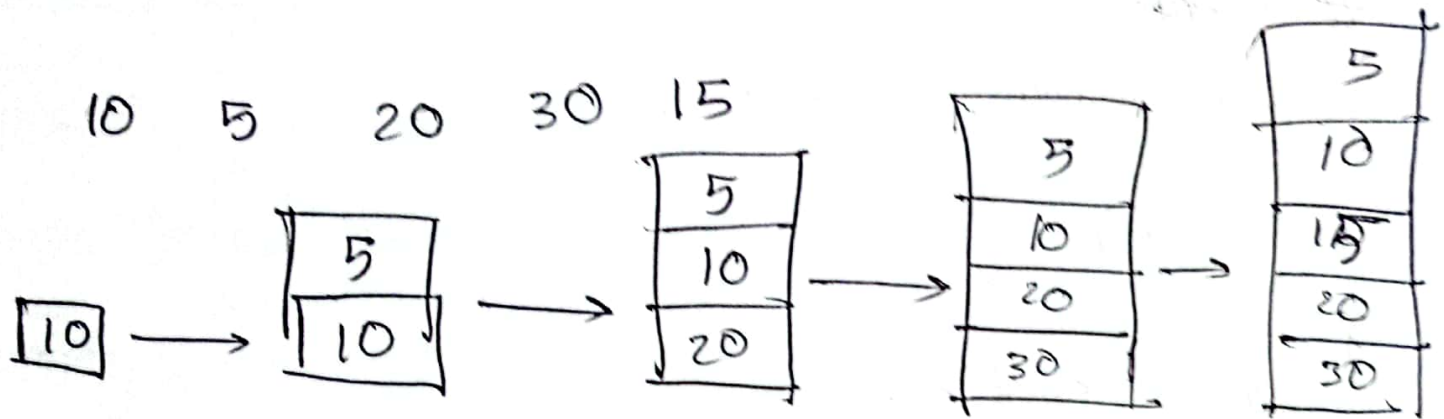
{ int val = s.front();

}

}

* Priority Queue

* setting condition for data insertion



priority - queue $\langle \text{int} \rangle s;$

for descending

$s.\text{push}(-\text{data})$

$\text{cout} << -\text{val} << \text{endl};$

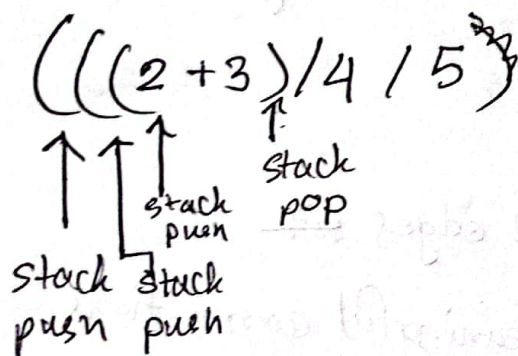
for ascending

* Deque

* Not necessary

* Double ended removal + add.

* Balanced Parentheses Problem (Stack)



If stack size $\neq 0$, not balanced parentheses

* Light Og

→ Discovering the web

Diagram illustrating a stack operation:

stack <char> temp

if ' ' {

→

L-7(W-7)

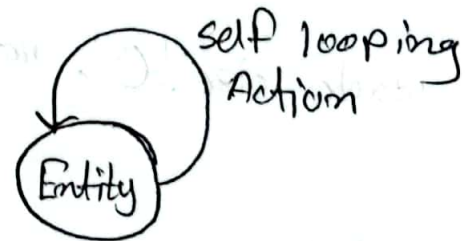
21/05/24

Graph $\rightarrow$ 30/100 in lab
 $\rightarrow$ 30/100 in theory



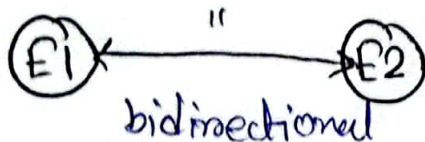
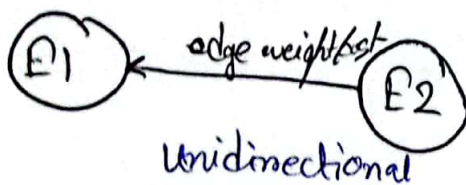
~~Whether~~ Entities must have meaningful connections.

Graph: * A group of nodes and edges with
* Nodes will have meaningful connections
* Connections are edges.
* Nodes > Edges



$\rightarrow$ At least 1 node is mandatory

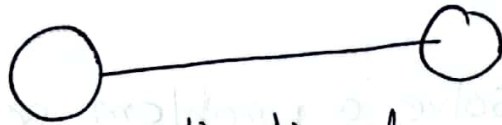
$\rightarrow$ Edges are not mandatory



Difference:
bidirectional can have 2 different weight
unidirectional will ALWAYS have same weight
* all bidirectional are unidirectional
but not all unidirectional are bidirectional



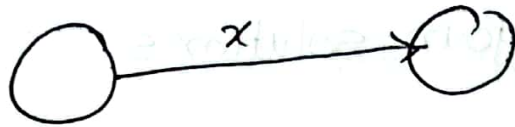
undirectional
unweighted graph



undirectional
unweighted graph



bidirectional
weighted graph



undirectional
weighted graph

* If any graph as at least one detached node, it is called disconnected graph

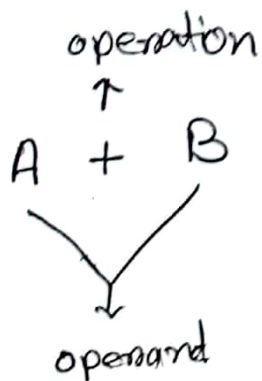
* Max edges : $\frac{n(n-1)}{2} \cdot [\text{Undirectional}]$

* Max edges in undirectional, undirectional and bidirectional?

Expression

(Part of stack)

An arithmetic expression can be represented in various forms such as prefix, infix or postfix. 'Pre', 'In', 'Post' refer to the relative position of the operator with respect to its operands. For example:



infix : $\langle \text{operand} \rangle \langle \text{operator} \rangle \langle \text{operand} \rangle$ $A + B$
 prefix : $\langle \text{operator} \rangle \langle \text{operand} \rangle \langle \text{operand} \rangle$ $+ AB$
 postfix : $\langle \text{operand} \rangle \langle \text{operand} \rangle \langle \text{operator} \rangle$ $AB +$

Easy for human
Hard for computer

$$6 + 2 * 7$$

Precedence

Associativity

(i) $() , \{ \} , []$

(ii) $\wedge$ R - L

(iii) $*$, $/$ L - R

(iv) $+$, $-$ L - R

Prefix :

$$* A * B + C / D$$

$$\Rightarrow *AB + C / D$$

$$\Rightarrow x + C / D$$

$$\Rightarrow x + /CD$$

$$\Rightarrow x + y$$

$$\Rightarrow + xy$$

$$\Rightarrow + *AB / CD \text{ (Ans)}$$

$$*1. a + b * c / d$$

$$2. 5 * (6 + 2) - (12 / 4)$$

$$1. a + b * c / d$$

$$\Rightarrow a + *bc / d$$

$$\Rightarrow a + x / d$$

$$\Rightarrow a + /xd$$

$$\Rightarrow a + y$$

$$\Rightarrow +ay$$

$$\Rightarrow +a /xd$$

$$= +a / *bcd \checkmark$$

$$2. 5 * (6 + 2) - (12 / 4)$$

$$\Rightarrow 5 * \{+62\} - \{/124\}$$

$$\Rightarrow 5 * a - b$$

$$\Rightarrow +5a - b$$

$$\Rightarrow c - b$$

$$\Rightarrow -bcb$$

$$\Rightarrow - *5ab$$

$$\Rightarrow - \frac{5}{1} \{+62\} \{/124\}$$

$$\Rightarrow - * 5 + 62 / 124$$

Postfix

$$* a + b * c / d$$

$$\Rightarrow a + bc * / d$$

$$\Rightarrow a + x / d$$

$$\Rightarrow a + y x d /$$

$$\Rightarrow a + y$$

$$\Rightarrow ay +$$

$$\Rightarrow a x d / +$$

$$\Rightarrow abc * d / + \checkmark$$

$$* 5 + (6 + 2) - (12 / 4)$$

$$= 5 * 62 + - 124 /$$

$$= 5 * x - y$$

$$= 5x * - y$$

$$= z - y$$

$$= zy -$$

$$= 5x * 124 / -$$

$$= 562 + * 124 / -$$

$$* A * B + C / D$$

$$\Rightarrow AB * + C / D$$

$$\Rightarrow x + C / D$$

$$\Rightarrow x + y$$

$$\Rightarrow xy +$$

$$\Rightarrow AB * CD / +$$

$$* K + L - M * N / P \wedge Q$$

$$\Rightarrow K + L - M * N / \frac{QP \wedge}{a}$$

$$\Rightarrow K + L - M * N / a$$

$$\Rightarrow K + L - \frac{MN *}{b} / a$$

$$\Rightarrow K + L - b / a$$

$$\Rightarrow K + L - \frac{ba}{c}$$

$$\Rightarrow K + L - c$$

$$\Rightarrow \frac{KL +}{d} - c$$

$$\Rightarrow d - c$$

$$\Rightarrow dc -$$

$$\Rightarrow KL + ba / -$$

$$\Rightarrow KL + MN * QP \wedge / -.$$

C-9(w-8)

28/05/24

Tower of Hanoi

- Three towers fixed
- all discs from source to destination
- at any time, no large disc on top of smaller ones.

Answer steps:

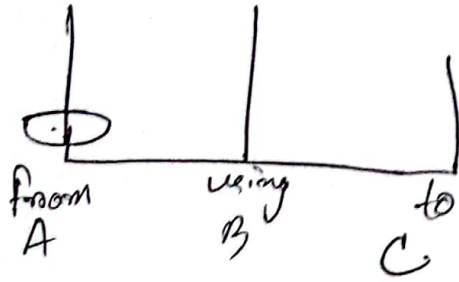
1. A to C
2. A to B
3. C to B
4. A to C
5. B to A
6. B to C
7. A to C



$$n = 3$$

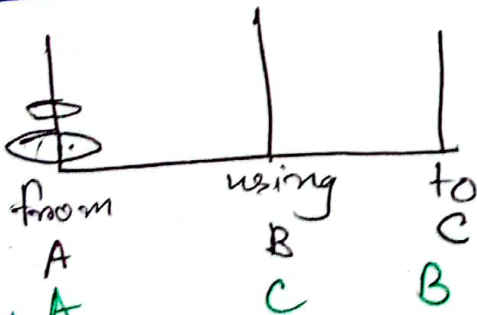
$$2^n - 1 = 2^3 - 1$$
$$= 7$$

Solution for 1 disc ($n=1$):



- Move a disc from A to C

Solution for 2 disc ($n=2$):



first one

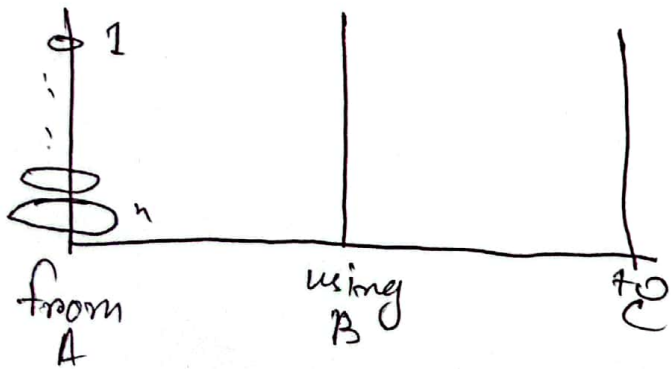
- Move ¹ disc from A to B using C
- Move disc from A to C
- Move disc from B to C using A

Solution for 3 discs ($n=3$):



- Move 2 discs from A to B using C
- Move a disc from A to C
- Move 2 discs from B to C using A

Solution of n discs:



- Move $(n-1)$ disc from A to B using C
- Move a disc from A to C
- Move $(n-1)$ discs from B to C using A

Code: disc no. pillars/towers

```
void TOH (↑n, A, B, C) // no. of discs, from, using, to)
{
```

~~TOH~~

```
if (n > 0)
{
```

```
    TOH (n-1, A, C, B)
```

print A to C // A to C will be changed depending on the recursion

```
    TOH (n-1, B, A, C)
```

```
}
```

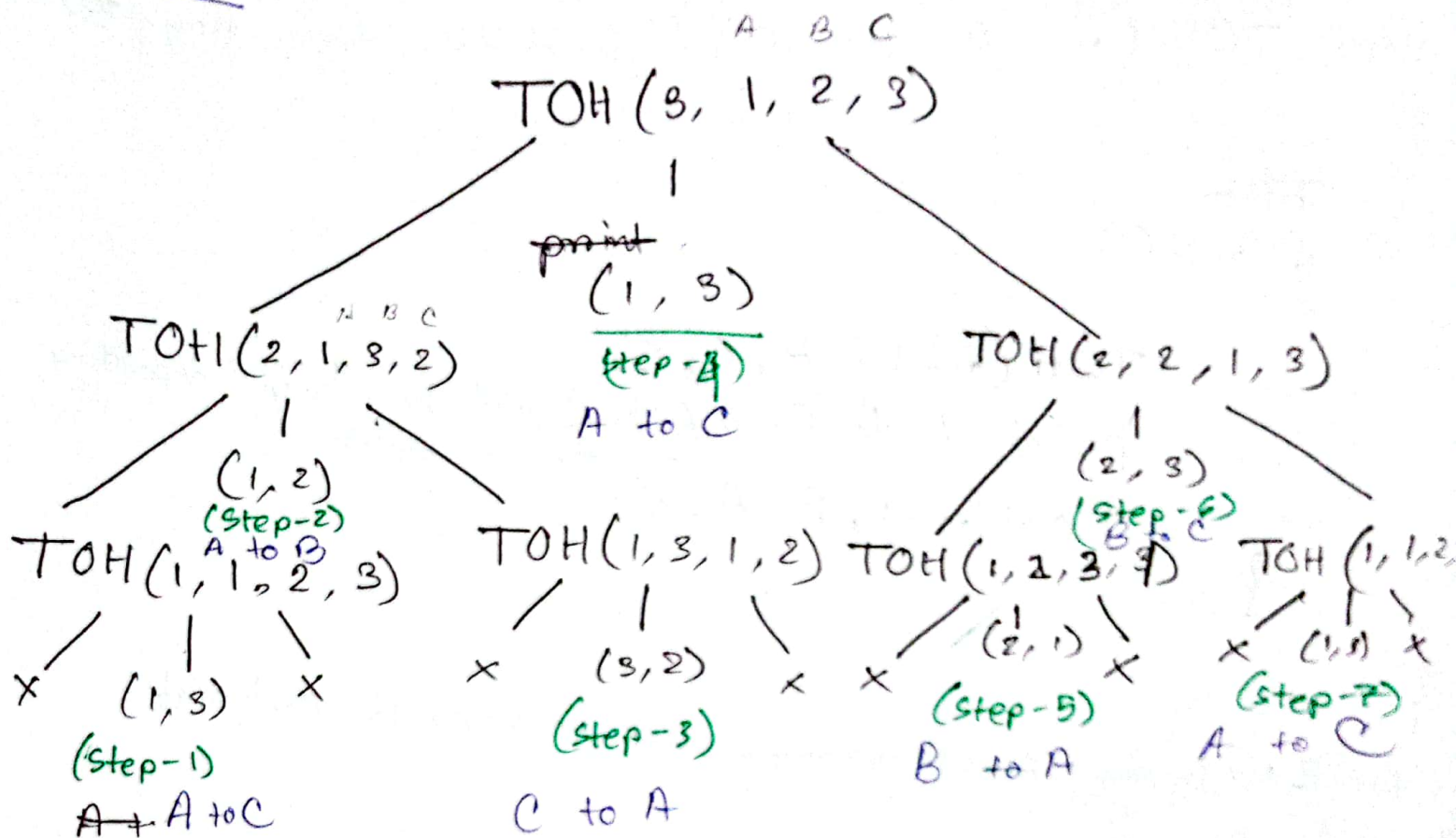
```
}
```

// for ~~DM~~ Dynamic ~~Arr~~ Programming, use an array for each state
Complexity: 2^n

* zobayer.blogspot.com - I, Me and Myself
- Attacking Recursion

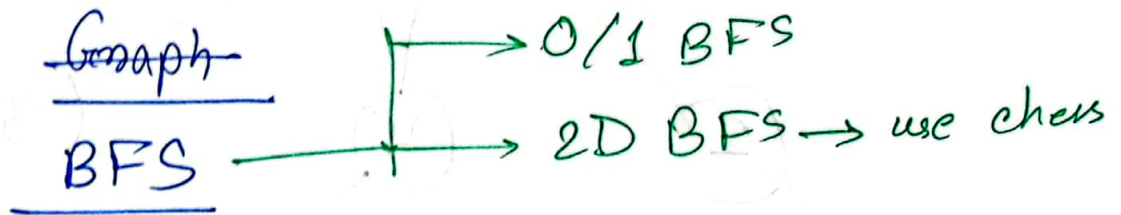
* Do TOH for 4 towers

Execution:



L-8(w-9)

04/06/24



(Breadth First Search)

- We have a graph: unweighted // all weights are equal
- we assume that all edges have equal/same weight
- Task: We have to find the shortest path length/path itself

from a source node to a destination node.

// Try to find all possible paths

BFS Procedure:

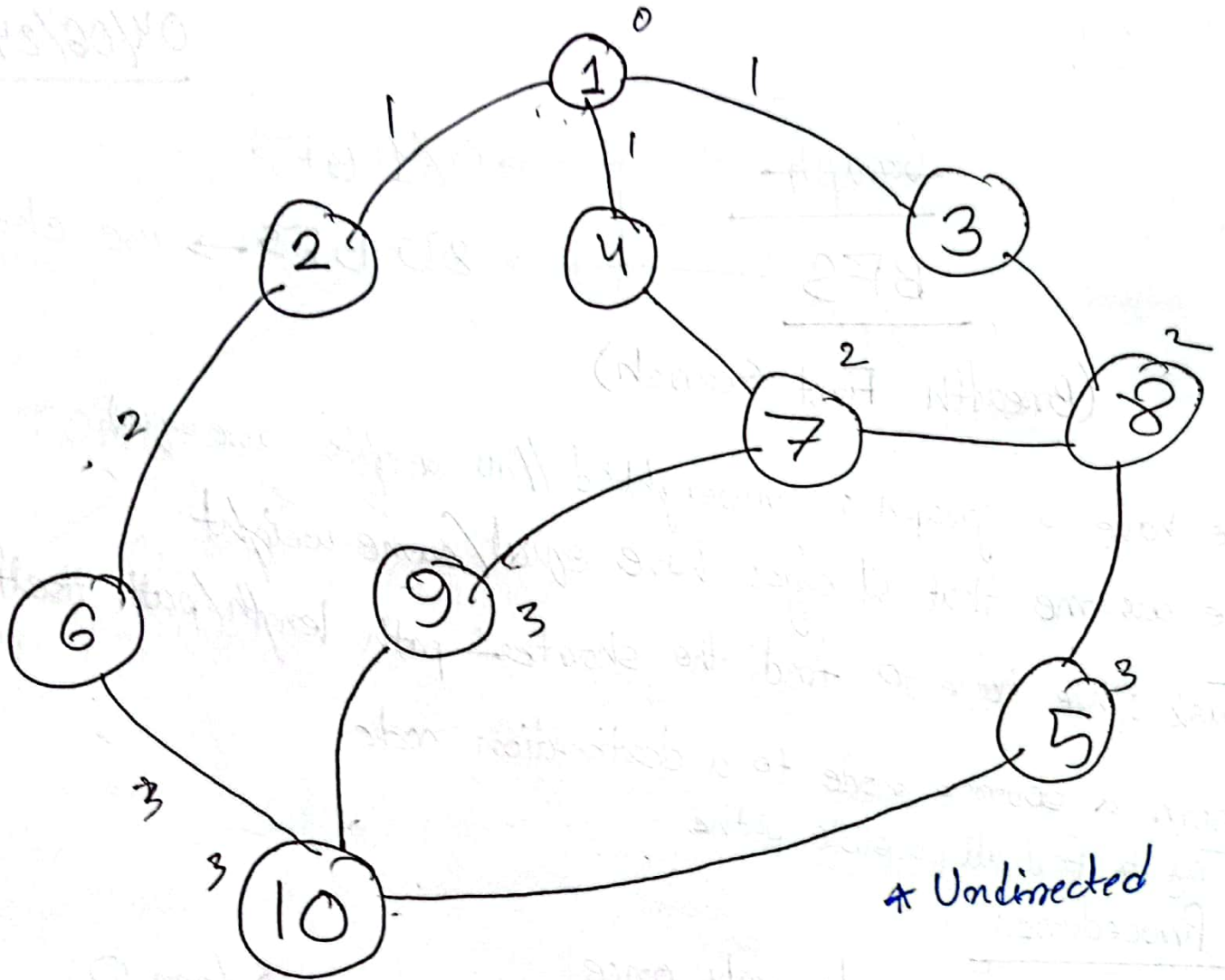
- we visit a node only once
- we assume that the source node is in level 0/gen 0
- All the nodes reachable from source nodes are in level 1;
- all the nodes reachable from level 1 are in level 2 and so on...

* Can find the length with path

* Path: All nodes passed through to reach a destination

* Considers everything other than source as destination

- Considers each path as unique



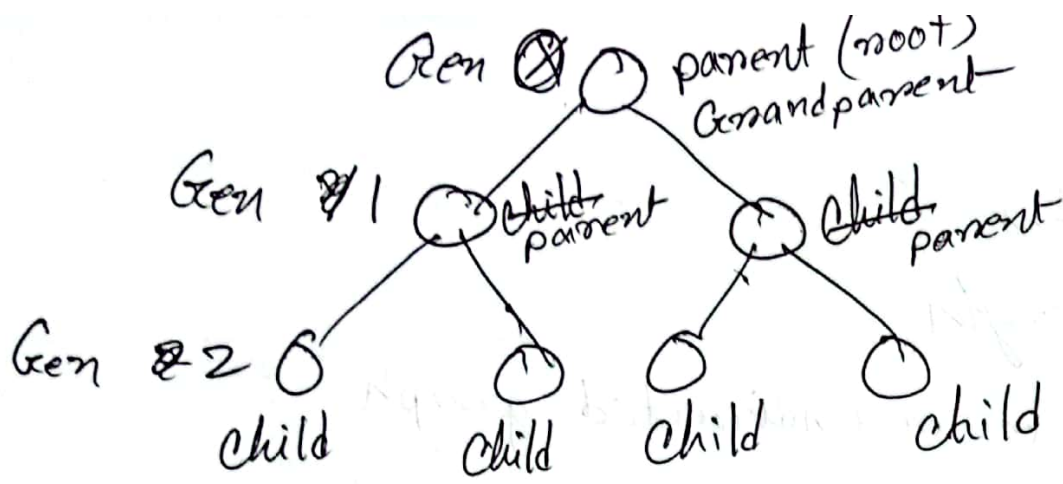
Input

10	13
1	2
1	4
1	3
2	6
4	7
3	8
3	7
6	10

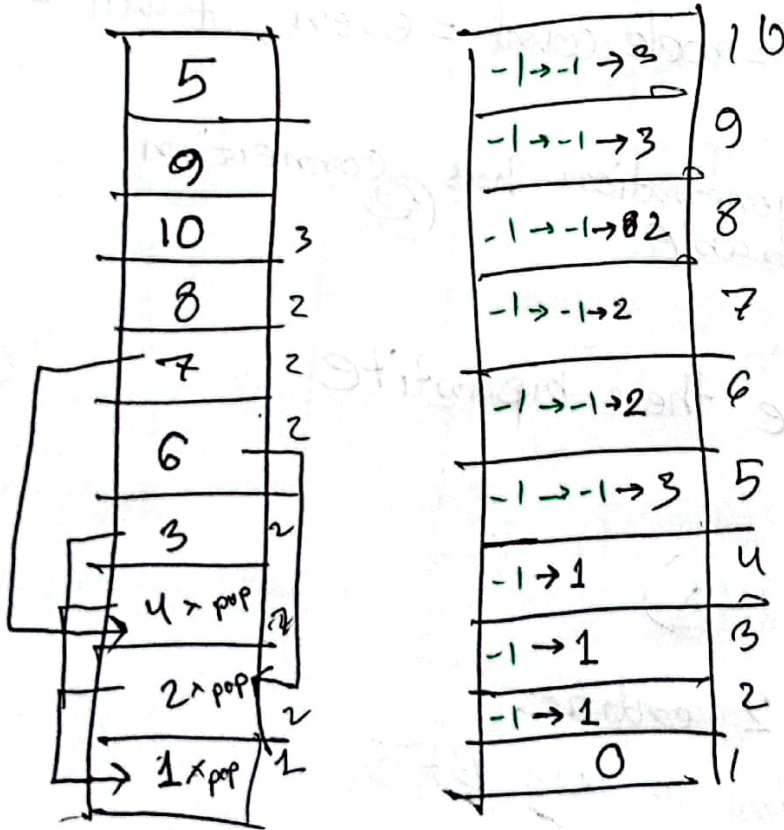
9	10
9	7
7	8
8	5
10	5

Adjacency list

1	→	2, 4, 3
2	→	1, 6
3	→	8, 1, 8, 7
4	→	1, 7
5	→	8, 10
6	→	10, 2, 10
7	→	4, 3, 9, 8
8	→	3, 7, 5
9	→	10, 7
10	→	6, 9, 5



- Find Generation
- Same generation will not have multiple parents



Queue

* BFS solves :

- Finds shortest path
- Finds distance length
- Cycle detection in an undirected graph
 - ↳ if edge count reduces, there is a cycle
(Learn)
- Bi-colorable
 - ↳ Cycle ~~edge count = node count~~ in cycle
 - ↳ if edge count = node count = even, it will be bicolourable
 - ↳ if nodes in same generation has connection then not bicolourable
- Bi-partition
 - ↳ if bicolourable then bipartite
- Commandos (Light OJ, UVA)
- Any problem involving 2 entities
 - ↳ try to use BFS

* MEGA MEAT MAN

- * Farthest node in a tree — (I) and (II)
- * Going Together

Algorithm :

0. BFS (Source)

1. Declare a queue

2. Enqueue source

3. level[source] = 0

4. while queue is not empty

5. u = deque

6. remove u from queue

7. For all adjacent nodes of u: // for adjacency

8. v = adjacent node of u // v is child, u is parent

9. if level[v] = -1

10. level[v] = level[u] + 1

11. Enqueue v

12. end if

13. end for

14. end while

Complexity : $O(\text{node} + \text{edge}) = O(n + m)$

Code:

~~0.~~ void

int level[2];

0. void BFS(int source) {

1. queue <int> Q;

2. Q.push(source);

3. level[source] = 0;

4. while (!Q.empty()) {

5. int u = Q.front;

6. Q.pop();

7. for(int j = 0; j < Node_Vec[u].size(); j++) {

8. int v = Node_Vec[u][j];

9. if (level[v] == -1) {

10. level[v] = level[u] + 1;

11. Q.push(v);

}

}

}

}

C-10 (w-9)

04/03/24

Infix to Postfix
using Stack

$$A * (B + C) - (D / E)$$

Pre: ~~$A * x - y$~~ $A * (+BC) - (/DE)$

$$= A * x - y$$

$$= * Ax - y$$

$$= z - y$$

$$= -zy$$

$$= - * Ax y$$

$$= - * A + BC / DE$$

Post: $A * (B + C) - (D / E)$

$$= A * (BC +) - (DE /)$$

$$= A * x - y$$

$$= Ax * - y$$

$$= z - y$$

$$= zy -$$

$$= Ax * y -$$

$$= ABC + DE / -$$

* Infix to Postfix :

$$A * (B + C) - (D / E)$$

For negative numbers:

convert $-x$ to

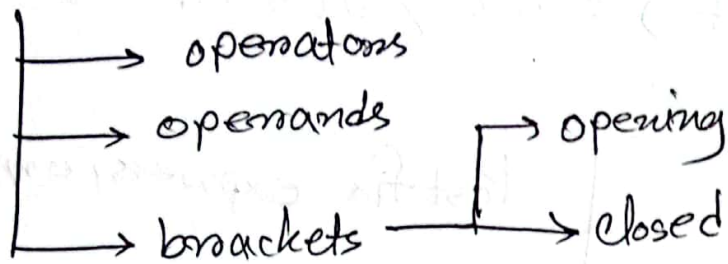
$$(-1 * x)$$

Symbol scanned	Stack	Postfix
A		A
*	*	A
(	*, (^{↓ Top}	A
B	*, (	AB
+	*, (, +	AB
C	*, (, +	ABC
)	* , + ^{↓ Top}	ABC +
-	-	ABC + * ABC + *
(	(, -, (	ABC + *
D	-, (, (	ABC + * D
/	-, (, /	ABC + * D E
E	-, (, /	ABC + * D E /
)	-	ABC + * D E / -

→ lower precedence

Symbol Table

options:



Precedence	
parentheses	↑
*, /	
+, -	

Conversion: → write for prefix

1) If symbol = operand, add to postfix

2) If symbol = '(', push to stack

3) If symbol = operator, check top of stack

iv) (i) if precedence (symbol) $\geq$ precedence (stack top),
push symbol to stack.

(ii) Otherwise, pop one by one from stack (operators)
and add to post fix until

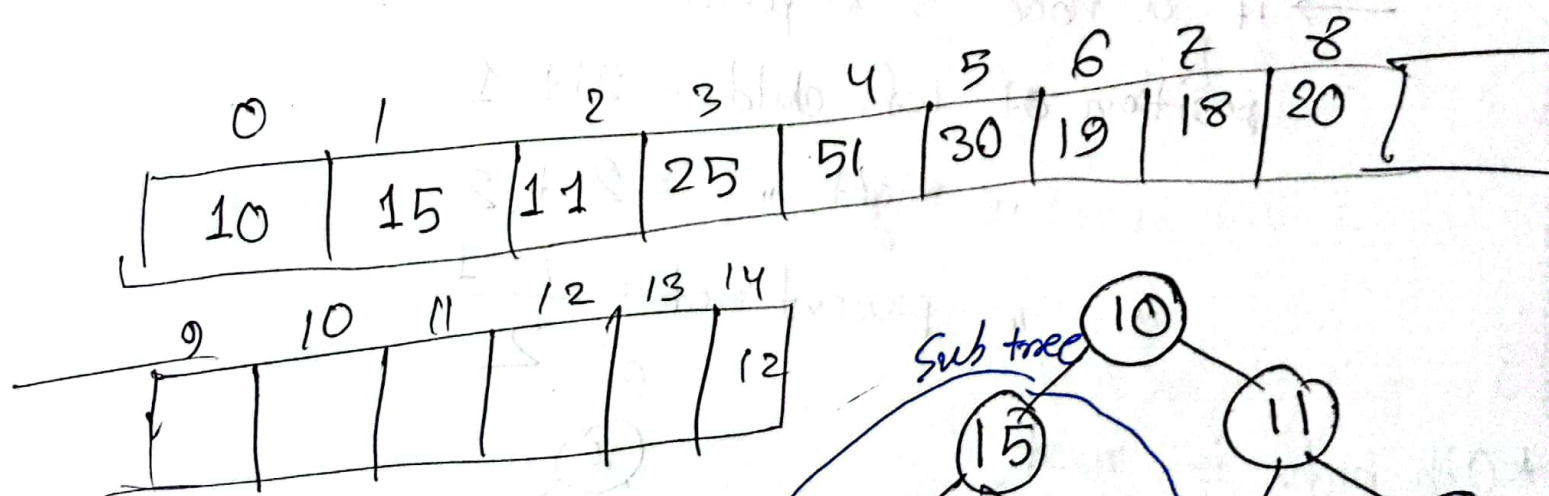
4) If symbol = ')', pop one by one, '(', is ~~or~~ add to
'post fix'

(otherwise (other solutions are available))

$$(A + B / C) * (D + E) - F = A B C / D E + * F -$$

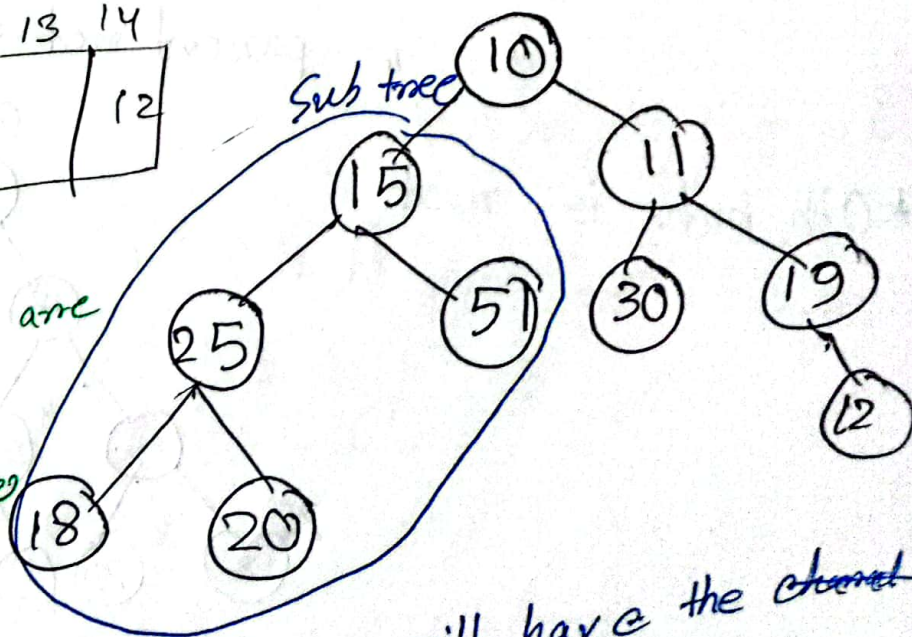
Symbol	Scan	Stack	Postfix expression
(		(	
A		(	A ✓
+		(	AB ✓
B		(, +	AB ✓
/		(, +	AB ✓
C		(, +, /	ABC ✓
*		(, +, /, *	ABC /
(		(, +, /, *, (	ABC /
D		(, +, /, *, (	ABC D
+		(, +, /, *, (, +	ABC D
E		(, +, /, *, (, +	ABC D E
)		(, +, /, *	ABC D E +
-		(, +, -	ABC D E + *
F		(, +, -, F	ABC D E + * F
)		-	ABC D E + * F -

* A tree is a finite collection of nodes that reflects one to many relationship among the nodes. An ordered tree has a specially designated node called root node. The nodes of a level are connected to the nodes of the upper level. The node that has no child is called leaf node. A node with a child node is called parent node.



* All nodes non-leaf nodes are parent nodes

* Basic tree = m-ary trees



* All sub trees will have the characteristics of parent tree

* Binary Tree: A finite set of nodes with a root node and every node has at most 2 child.

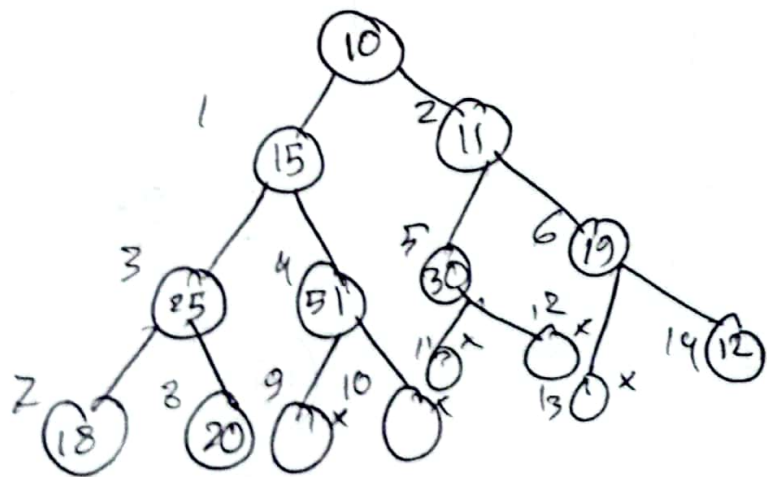
In a binary tree, // For Array

→ if no. of levels = k then,
maximum number of nodes in tree, $n = 2^k - 1$

→ if maximum no. nodes = n then,
no. of levels in tree, $k = \lceil \log_2(n+1) \rceil$

→ if a node is in position = i then,
position of left child = $2i + 1$
" " right " = $2i + 2$
" " parent node = $\frac{i-1}{2}$

* 0th index is root



* Doubly linked list can also make binary tree

* Tree Traversal Technique:

(a) Pre - Order

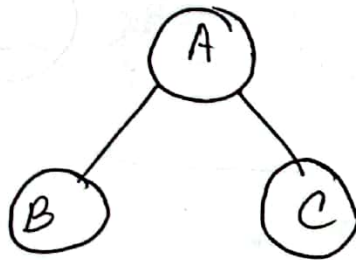
root
left
right

(b) In - Order

left
root
right

(c) Post - Order

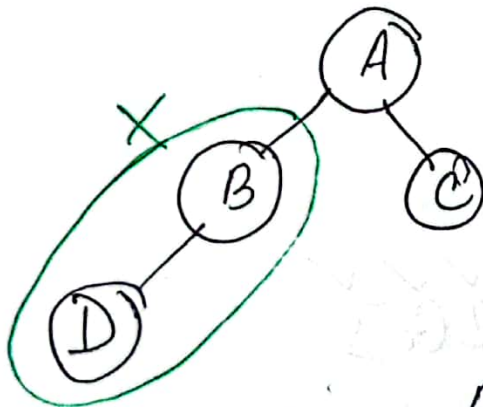
left
right
root



Pre : ABC

In : BAC

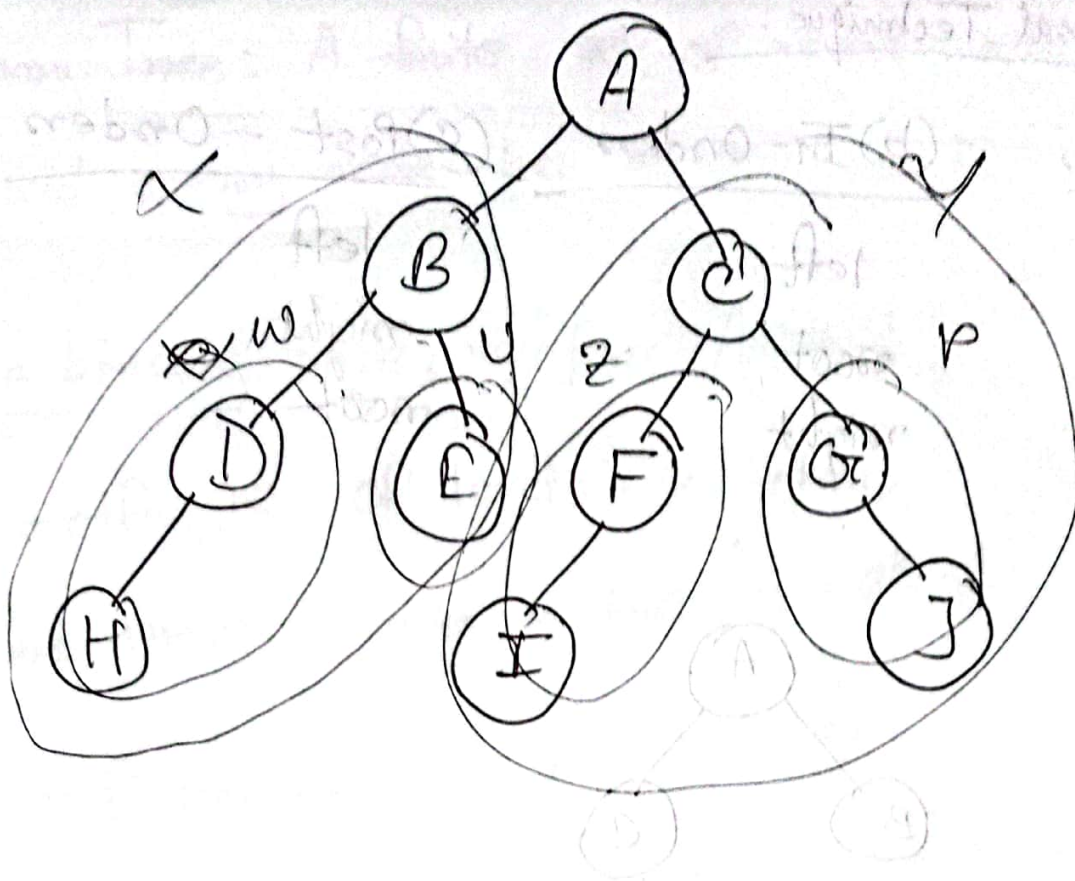
Post : BCA



Pre-order : $A \times C = A B D C$

In - order :

Post - order :



Pre : AXY

= ABWUCZP

= ABDHECFIGJ

Post In : XAY

= WBUAZCP

= HDBEAIFCGJ

Post : XYA = WBDEAIFJGCA

* Algorithm:

0. Pre-order (temp):

1. if temp $\neq$ NULL:
2. print temp $\rightarrow$ data //
3. pre-order (temp $\rightarrow$ left) // print in in-order
4. pre-order (temp $\rightarrow$ right) // print in post-order
5. end if

* For array implementation $\rightarrow$ send index

* Root is printed directly

Complexity: $O(n) = \log_2(\text{node})$

* Categories of Binary Tree:

(a) Rooted Binary

(a) Rooted Binary Tree:

Every binary tree is a rooted binary tree.

(b) Full Binary Tree:

A binary tree in which every node has either

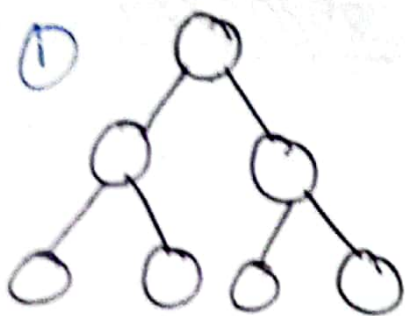
0 or 2 nodes.

(c) Perfect Binary Tree:

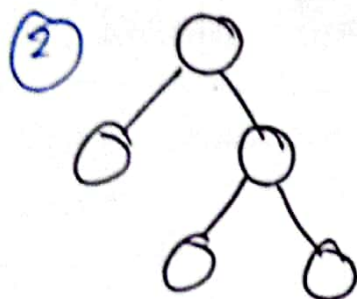
A binary tree in which all internal nodes have 2 children all leaves are in the same level.

(d) Complete Binary Tree:

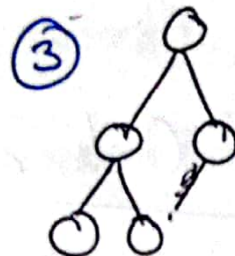
A binary tree in which every level except possibly the last is completely filled and nodes in the last level are as far LEFT as possible



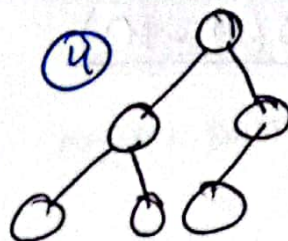
- Rooted
- Perfect
- Full
- Complete



- Rooted



- Rooted



- Rooted

- Full

- Full

- Complete
- Complete

L-9(w-10)

Online

25/06/24

See question

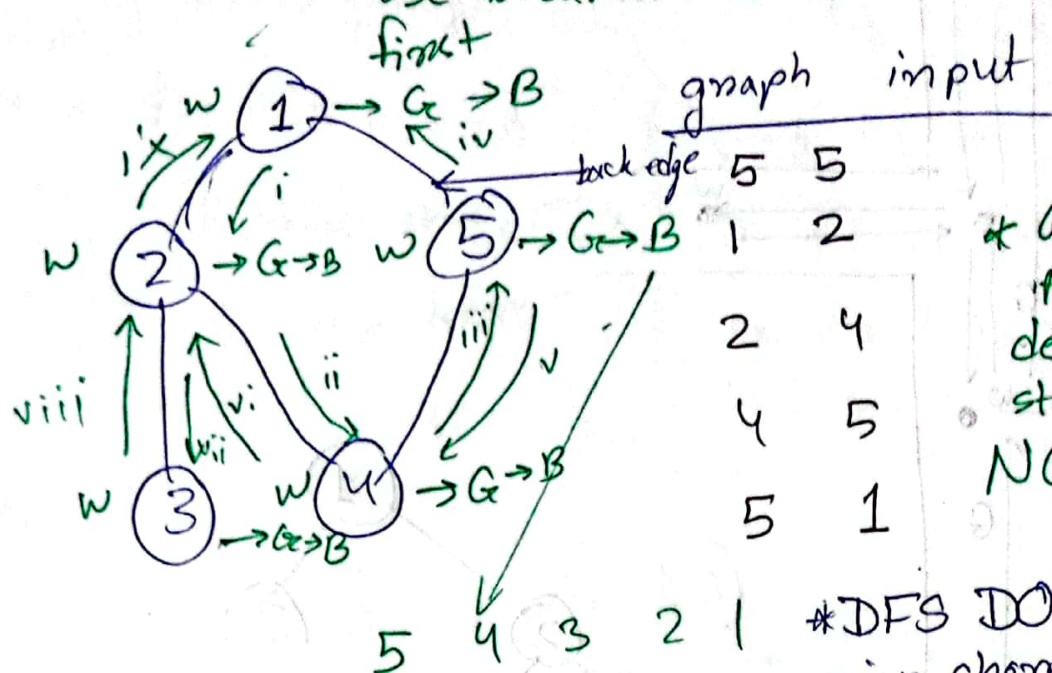
```
for (i = 1; i <= node_no; i++)  
{  
    if (level[i] == -1)  
    {  
        BFS(i);  
        cout << i << endl;  
        k++;  
    }  
}
```

HEAVILY

Depends on user input

DFS (Depth First Search)

Use recursion to reach the deepest point



* Going back previous node depends on stack memory. NOT EDGE.

* DFS DOES NOT give shortest path if back edge is

Three Colors:

white = node not visited yet

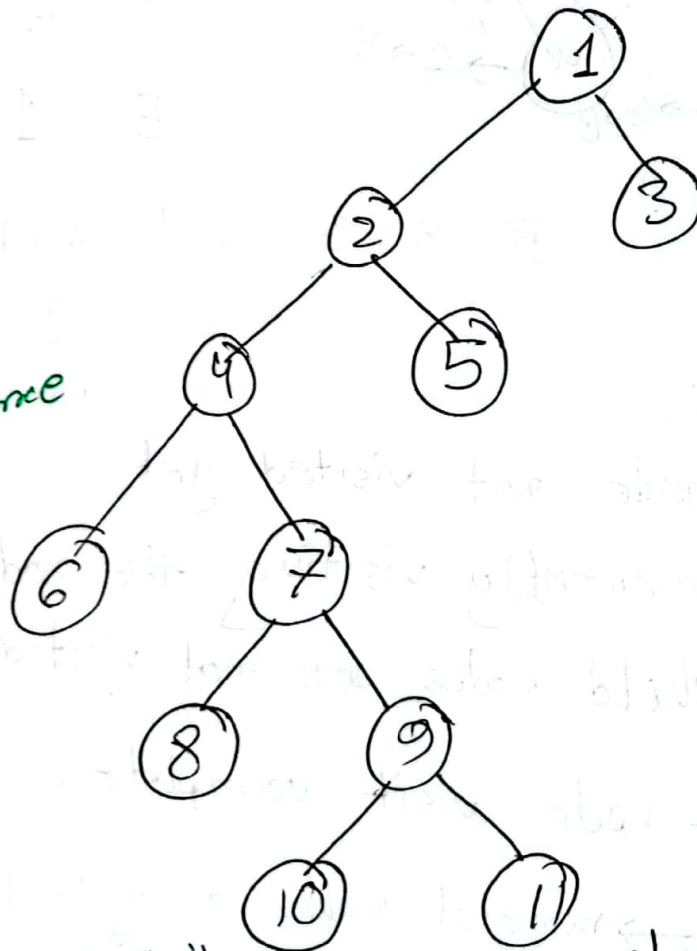
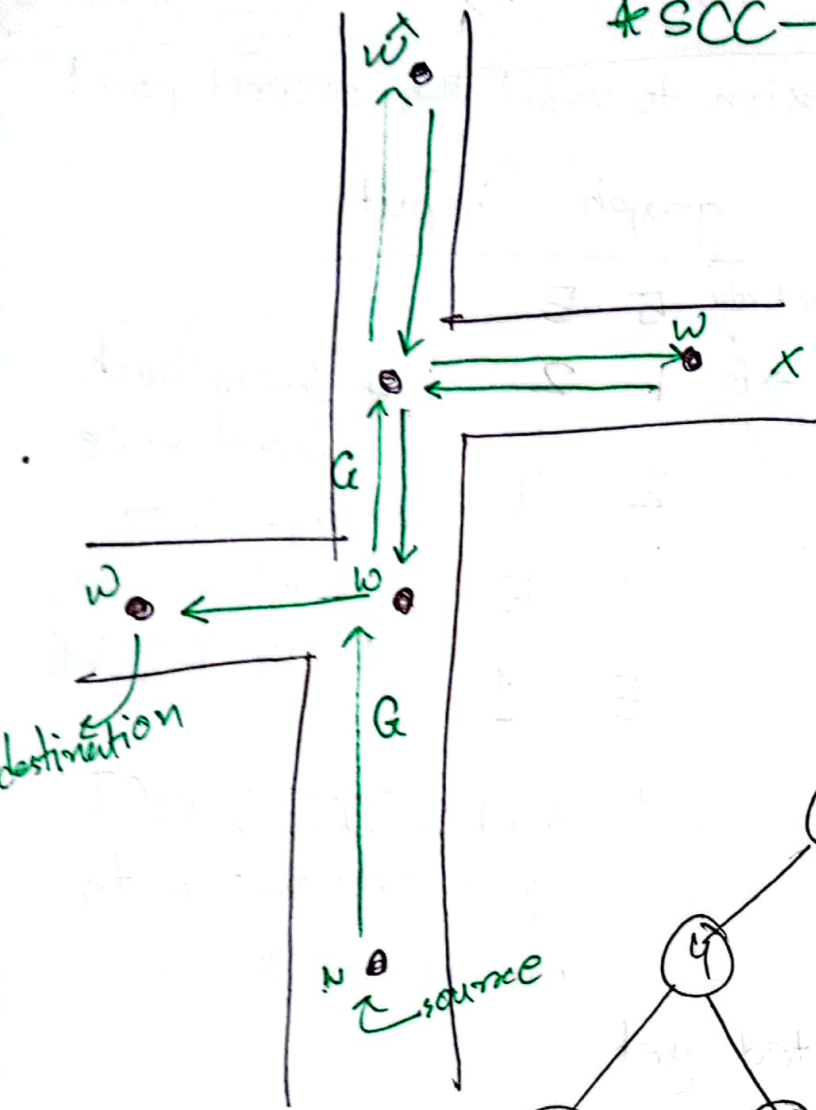
gray = currently visiting the node but all the child nodes are not visited yet

black = node visit complete.

* back edge → removal makes a cycle-less tree. graph/tree

* tree edge.

* SCC $\rightarrow$ Used to find numbers of cycles.



* graph traversal

* cycle finding $\rightarrow$ when $g = g$, it has cycle

* cycle / has cycle graph finding

- * connected/disconnected graph finding white
- * count numbers of components in a graph $\rightarrow$ Number of times comp is run BFS/DFS

- * Topological sort

* Topological Sort:

In a ^{directed} dependent graph, it will give a sorted step.

- Multiple solutions are possible
- Run DFS with the least in orders.

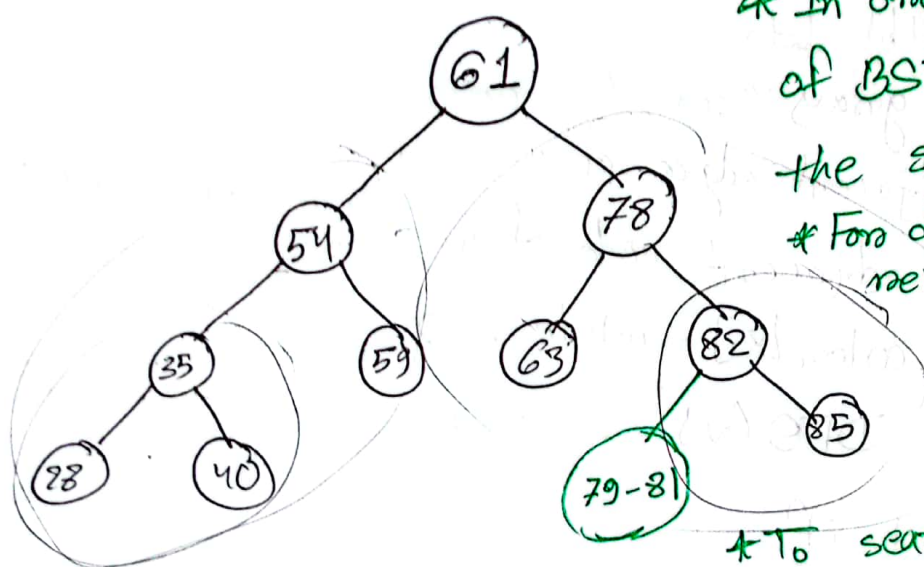
DFS Algorithm:

0. DFS(u) // initial data
1. color[u] = gray // set it to gray
2. For all adjacent edges of u: // Same loop as BFS
3. v = adjacent node of u
4. if color[v] = white: //
5. DFS(v) // recursion
6. end if
7. end For
8. color[u] = black // when done make it black

Complexity: $O(\text{node} + \text{edge})$

Binary Search Tree (BST)

A binary search tree is a binary tree where ^{all} the node values of the left sub tree are smaller than the node value of the root and all the node values of the right sub tree are greater than the node value of the node



* In order traversal of BST is sorts the set

* For descending order, revert the tree

* To search, we pre-order algorithm then use if switch to check

Inorder: X 61 Y

= W 54 59 61 63 78 V

= 28 35 40 54 59 61 63 78 82 85

Search Code :

```
int val, flag = 0;
```

```
void pre_order(node *temp)
```

```
{
```

```
    if (temp != NULL)
```

```
    {
```

```
        if (temp->data == val)
```

```
        {
```

```
            flag = 1;
```

```
        }
```

```
    else if (temp->data > val)
```

```
    {
```

```
        pre_order(temp->left) // For greatest smallest ignore right this
```

```
    }
```

```
    else
```

```
    {
```

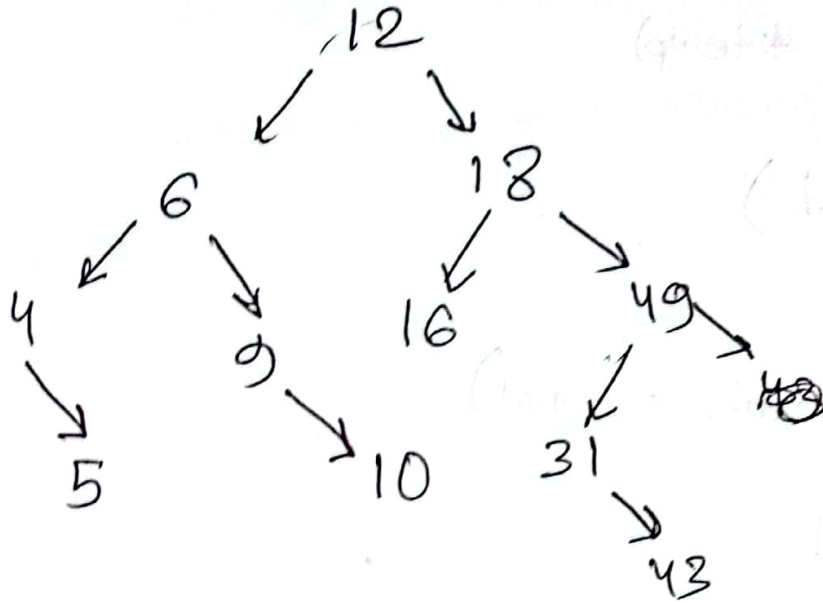
```
        pre_order(temp->right) // For smallest ignore this
```

```
    }
```

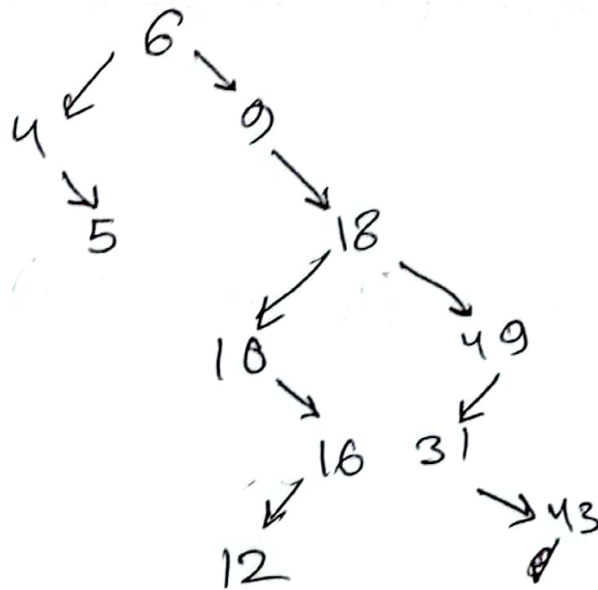
```
}
```

```
}
```

12, 6, 9, 18, 4, 10, 5, 16, 49, 31, 43



Now 12 at last,



Binary Search Tree Code: $\rightarrow$ Complexity: $O(n) = n \log_2 n$

struct node

{

int data;

node *left;

node *right;

};

node *root = NULL

void BST insert(int val)

{

node *temp;

temp = new node();

temp $\rightarrow$ data = val;

temp $\rightarrow$ left = NULL;

temp $\rightarrow$ right = NULL;

if (root == NULL) { root = temp; }

else

{ node *curr_node = root;

node *prev_node = root;

while (curr_node != NULL)

{ prev_node = curr_node;

if (curr_node $\rightarrow$ data > val) { curr_node $\rightarrow$ left; }

else { curr_node = curr_node $\rightarrow$ right; }

if (prev_node $\rightarrow$ data > val) { prev_node $\rightarrow$ left = temp; }

else { prev_node $\rightarrow$ right = temp; }

}

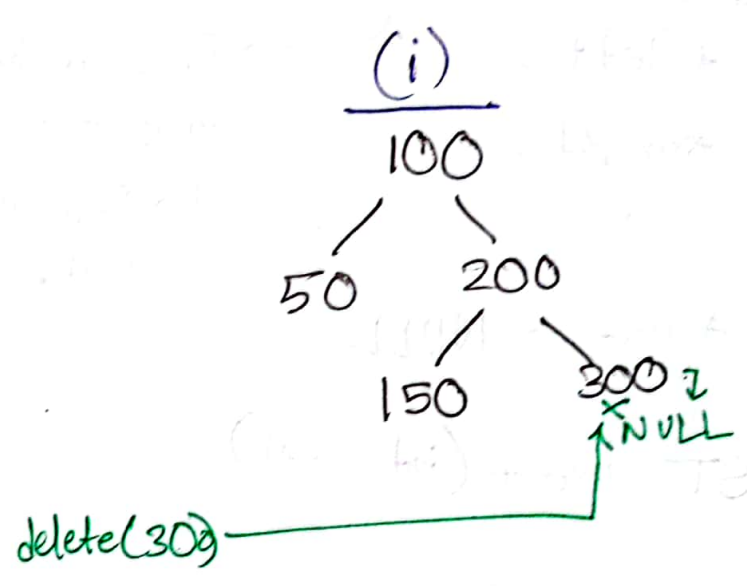
* predecessor

* successor

Delete From a Binary Search Tree

Tree cases:

- (i) leaf node
- (ii) node with one child
- (iii) node with two child



[Faint handwritten notes and code snippets are visible in the background, including:]

```

if (root == NULL) return root;
if (root->data == key) {
    // Case 1: Leaf node
    if (root->left == NULL & root->right == NULL) {
        delete root;
        return NULL;
    }
    // Case 2: Node with one child
    if (root->left != NULL & root->right == NULL) {
        return root->left;
    }
    if (root->left == NULL & root->right != NULL) {
        return root->right;
    }
    // Case 3: Node with two children
    // Find inorder successor
    Node* succ = root->right;
    while (succ->left != NULL) succ = succ->left;
    // Replace root with succ
    root->data = succ->data;
    // Delete succ
    succ->data = key;
    return delete(succ);
}
// Case 4: Node is in left subtree
root->left = delete(root->left, key);
return root;
// Case 5: Node is in right subtree
root->right = delete(root->right, key);
return root;
    
```

```
void delete_node(int val)
```

```
{
```

```
    node *curr_node = root;
```

```
    node *prev = root;
```

```
    while (curr_node != NULL)
```

```
    {
```

```
        if (curr_node->data == val) {
```

```
            break;
```

```
        }
```

```
        if (curr prev
```

```
            prev = curr_node;
```

```
        if (curr_node->data < val) {
```

```
            curr_node = curr_node->right;
```

```
        }
```

```
        else {
```

```
            curr_node = curr_node->left;
```

```
        }
```

```
    }
```

```
    if (curr_node == NULL) {
```

```
        return;
```

```
    else {
```

```
        if (curr_node->right == NULL) && (curr_node->left == NULL)
```

```
        {
```

```
            if (prev->right == curr_node)
```

```
            {
```

```
                prev->right = NULL;
```

```
            } else {
```

```
                prev->left = NULL;
```

```
            }
```

case (i)
on leaf

Does not activate until
value is found

used to navigate

else if (curr_node → right == NULL || curr_node → left == NULL)

}

node *child; // temp node

if (curr_node → right == NULL)

{

child = curr_node → left;

}

else

{

child = curr_node → right;

}

if (prev → right == curr_node) // Find curr
wrt prev

{

prev → right = child;

}

else

{

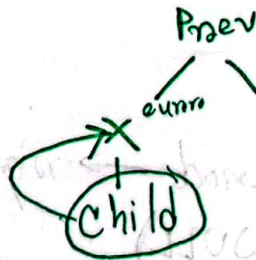
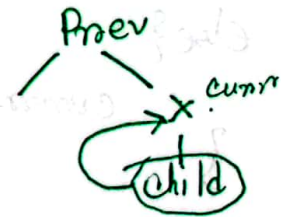
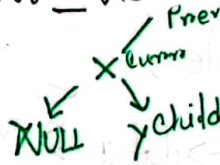
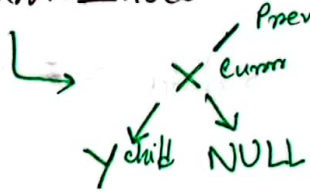
prev → left = child;

}

}

}

Set child



case
(i)
↓
one child?

```

else
{

```

```

node *min = curr_node;
min = curr_node; prev = curr_node;
while (min->left != NULL)
{

```

```

min = min->left;

```

```

}

```

```

node *temp;

```

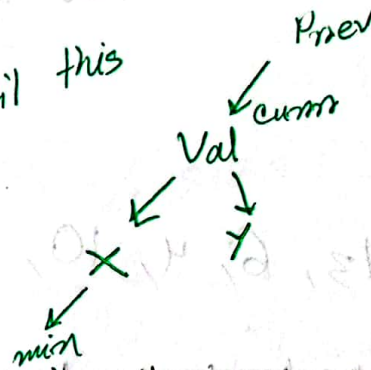
```

temp = min

```

* Find Do the same with right max value.

Until this



```

else
{

```

```

node *temp = curr_node;

```

```

prev = curr_node;

```

```

curr_node = curr_node->right;

```

```

while (curr_node->left != NULL)
{

```

```

prev = curr_node;

```

```

curr_node = curr_node->left;

```

```

}

```

```

temp->data = curr_node->data; // swaps Val with the min data

```

```

if (prev->left == curr_node)
{

```

```

prev->left = curr_node->right;

```

```

}

```

```

else
{

```

```

prev->right = curr_node->right;

```

```

}

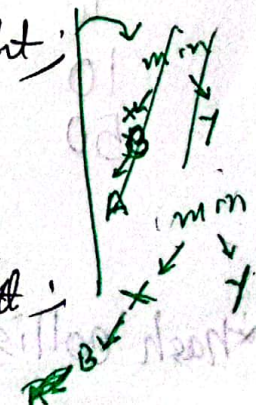
```

case (iii)

two children



Finding minimum of



C-14 (W-13)

Hashing (10 from here)

15/07/24

~~100~~

Linear search — $O(n)$

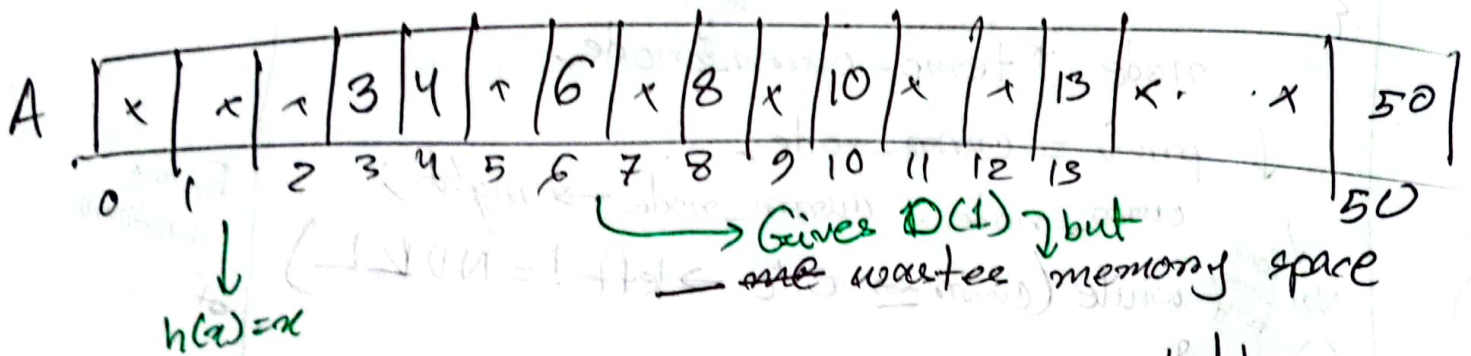
Binary Search — $O(\log_2 n)$

→ Different from cryptography
Hashing is more involved with searching

→ Does not give $O(1)$ but does work better than $O(n)$ and $O(\log_2 n)$

We want a search method that requires $O(1)$ time

keys = 8, 3, 13, 6, 4, 10, 50



key space

8
3
13
6
4
10
50

hashing function

$$h(x) = x \% 10$$

hash table

	9
8	8
	7
6	6
	5
4	4
3	3
	2
	1
10	0

*hash collision: conflict for space.

hash collision

- linear probing
- quadratic probing
- separate chaining

Linear Probing

$$h(x) = x \% \text{size}$$

$$h'(x) = [h(x) + f(i)] \% \text{size}$$

$$f(i) = i ; i = 0, 1, 2, \dots$$

$$* \text{ search} = A[x] \rightarrow A[x+1] \rightarrow \dots$$

$$h(13) = 13 \% 10 = 3 \rightarrow \text{collision}$$

$$\therefore h'(13) = (13 + 1) \% 10 = \frac{4 \cdot 10}{3+1} = 4 \rightarrow \text{collision}$$

$$\therefore h'(13) = (13 + 2) \% 10 = \frac{5 \cdot 10}{3+2} = 5 \rightarrow \text{free}$$

Quadratic Probing

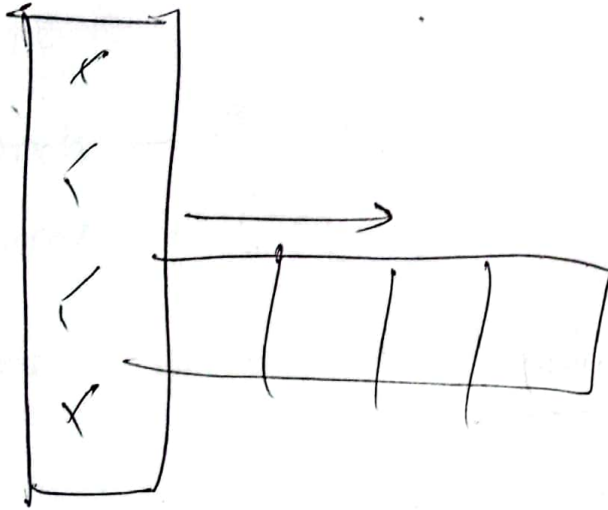
$$h(x) = x \% \text{size}$$

$$h'(x) = [h(x) + f(i)] \% \text{size}$$

$$f(i) = i^2 ; i = 0, 1, 2, \dots$$

~~Separ~~

Separate chaining



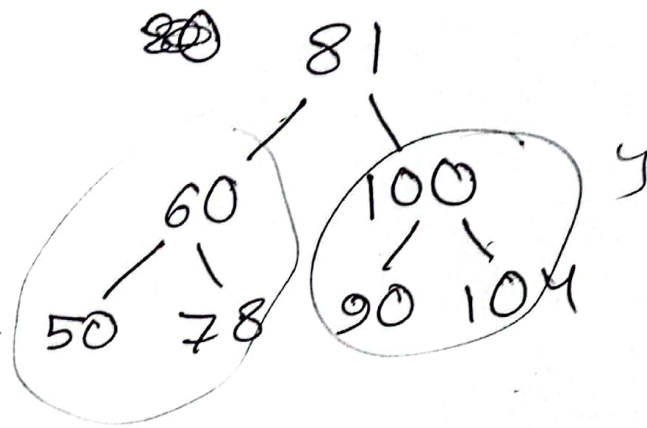
* 2D vector

Coalescent Probing

Coalescent Probing

1-10(w-13)

17/09/24



Pre: ~~x 81 y 81 x y~~ ~~x 81~~

~~= 81 w 60 v 100 p q~~

~~= 81 60 50 81 60 50 78 100 90 104~~

In: ~~x 81 y~~

~~= w 60 v 81 p 100 q~~

~~= 50 60 78 81 90 100 104~~

Post: ~~x y 81~~

~~= w v 60 p q 100 81~~

~~= 50 78 60 90 104 100 81~~